RÉPUBLIQUE TUNISIENNE
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
INSTITUT SUPÉRIEUR D'INFORMATIQUE ET DE MATHÉMATIQUES DE MONASTIR

Correction TP N° 01

Dans le fichier zip : on va lire tout d'abord « README »

Puis « requirements.txt »

1. Préparation avent la séance – installation :
   a. Installer les logiciels (sous Windows)
      1) Télécharger VsCode de « code.visualstudio.com »
      2) Dans notre machine windows on va Télécharger « python » de www.python.org
      3) Configurer PATH dans windows dans « Modifier les variables d'environnement », PATH , Modifier puis Nouveau et ajouter le path de python.exe
      4) Configurer PATH dans VsCode :
         - Ctrl+shift+p
         - Python : Select Interpreteur
         - OK
      5) Dans VsCode ; new Terminal :

python --version

   b. Ouvrir le projet dans VsCode : File -> Open Folder -> Lab01_Student_Package
   c. Créer un Environnement virtuel :

python -m venv .venv

- python : lance Python.
- -m venv : demande à Python de créer un environnement virtuel.
- .venv : nom du dossier dans lequel l'environnement sera créé.

   d. Activer l'Environnement virtuel

.\.venv\Scripts\Activate.ps1

```
PS C:\Users\yosra> .\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\yosra>
```

   e. Sélectionner l'interpreteur python
      - Ctrl+shift+p
      - Python : Select Interpreteur

6) Installer les Bibliothèques :

python -m pip install -r requirements.txt

1- Python : Indique à l'ordinateur d'utiliser Python pour exécuter une commande.
2- « -m pip » : Demande à Python d'exécuter le module pip, qui est l'outil utilisé pour installer des packages Python.
3- Install : Signifie « installer » les bibliothèques demandées.
4- « -r requirements.txt » : « -r » signifie que pip doit lire la liste des packages dans le fichier « requirements.txt », puis installer les packages qui y sont indiqués.

Que contient le fichier requirements.txt ?

- numpy >= 1.23
- pandas >= 1.5
- matplotlib >= 3.6
- Scikit-learn >= 1.2

Setup Cell :

```
import numpy as np

# Importe la bibliothèque NumPy sous l'alias « np ».

# NumPy permet de manipuler des tableaux numériques (arrays),

# d'effectuer des calculs mathématiques, des opérations matricielles et des traitements

# rapides sur les données.

import pandas as pd

# Importe la bibliothèque Pandas sous l'alias « pd ».

# Pandas permet de manipuler et d'analyser des données tabulaires.

# Ses principales structures sont Series (une dimension)

# et DataFrame (tableau avec lignes et colonnes).

import matplotlib.pyplot as plt

# Importe le module pyplot de Matplotlib sous l'alias « plt ».

# Il sert à créer des graphiques : courbes, histogrammes,

# nuages de points (scatter plots), diagrammes en barres, etc.

# Il permet de visualiser les données et les résultats des modèles.

from sklearn.datasets import load_iris, load_wine

# Importe deux fonctions de la bibliothèque scikit-learn.

# load_iris() charge le jeu de données Iris :

# 150 fleurs, 4 caractéristiques et 3 espèces.

# load_wine() charge le jeu de données Wine :

# 178 échantillons, 13 caractéristiques et 3 classes de vin.

# Ces datasets servent à expérimenter des algorithmes de classification.

from sklearn.model_selection import train_test_split

# Importe la fonction train_test_split.

# Elle divise les données en deux ensembles :

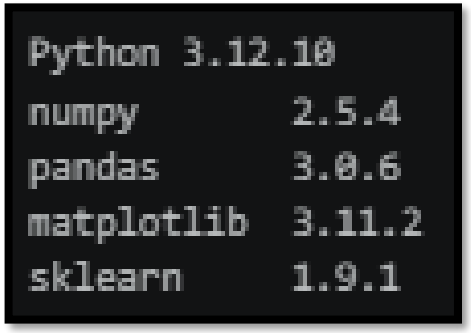
# - Train (entraînement) : pour apprendre le modèle.

# - Test : pour évaluer le modèle sur des données non vues.
```

```
# On peut utiliser stratify=y pour conserver les proportions
# des classes dans les deux ensembles.
from sklearn.dummy import DummyClassifier
# Importe DummyClassifier, un classificateur de référence (baseline).
# Il effectue des prédictions selon une stratégie simple,
# sans apprendre les relations entre les caractéristiques et la cible.
# Exemple : strategy="most_frequent" prédit toujours
# la classe la plus fréquente dans l'ensemble d'entraînement.
# Il sert à vérifier si un vrai modèle fait mieux qu'une méthode naïve.
from sklearn.neighbors import KNeighborsClassifier
# Importe l'algorithme k-Nearest Neighbors (kNN).
# Il classe un nouvel échantillon en examinant ses k voisins
# les plus proches dans l'ensemble d'entraînement.
# La classe prédite est généralement celle qui est majoritaire
# parmi ces voisins (avec weights="uniform", par défaut).
# Exemple : n_neighbors=5 signifie que kNN examine 5 voisins.
from sklearn.metrics import accuracy_score, confusion_matrix
# Importe deux fonctions permettant d'évaluer un classificateur.
# accuracy_score : calcule la proportion de prédictions correctes.
# Formule : Accuracy = nombre de prédictions correctes / nombre
total de prédictions.
# confusion_matrix : construit une matrice de confusion,
# qui compare les classes réelles aux classes prédites.
# Les lignes représentent les classes réelles
# et les colonnes représentent les classes prédites.
SEED = 42   # fixed seed: everyone in the room gets the same split
# Définit une graine aléatoire fixe égale à 42.
# Le nombre 42 est arbitraire : on aurait pu choisir un autre entier.
# En utilisant random_state=SEED dans train_test_split,
# on obtient la même répartition Train/Test à chaque exécution
# avec les mêmes données et les mêmes paramètres.
# Cela favorise la reproductibilité des expériences.
np.set_printoptions(precision=3, suppress=True)
# Configure l'affichage des tableaux NumPy.
# precision=3 : affiche les nombres flottants avec une précision
# de 3 chiffres après la virgule en affichage positionnel.
# suppress=True : évite la notation scientifique pour les
# petites valeurs lorsque cela est possible.
# Exemple : 0.0001234 sera affiché 0.000 plutôt que 1.234e-04.
# Cette instruction modifie l'affichage, pas les valeurs stockées.
```

Task1—Environment-check

```
import sys, sklearn, matplotlib
# Importe trois modules ou bibliothèques Python :
# - sys : module standard permettant d'accéder aux informations
#   sur l'interpréteur Python et son environnement d'exécution.
# - sklearn : bibliothèque scikit-learn, utilisée en Machine Learning
#   pour la classification, la régression, le prétraitement, etc.
# - matplotlib : bibliothèque utilisée pour créer des graphiques.
print("Python", sys.version.split()[0])
# Affiche la version de Python utilisée pour exécuter le programme.
# sys.version : retourne une chaîne contenant la version de Python
#                   ainsi que d'autres informations sur sa compilation.
# .split() : découpe cette chaîne en plusieurs éléments,
#            en utilisant les espaces comme séparateurs.
# [0] : sélectionne le premier élément, qui correspond
#       au numéro de version de Python.
# print() : affiche le texte "Python" suivi du numéro de version.
for m in (np, pd, matplotlib, sklearn):
    # Parcourt successivement quatre bibliothèques Python :
    # np        : NumPy, importée précédemment sous l'alias np.
    # pd        : Pandas, importée précédemment sous l'alias pd.
# matplotlib : bibliothèque de visualisation graphique.
# sklearn    : bibliothèque de Machine Learning.
# m est une variable qui représente la bibliothèque courante.
# La boucle s'exécute donc quatre fois.
print(f"{m.__name__:<11}", m.__version__)
# Affiche le nom et la version de chaque bibliothèque.
# f"..." : f-string, chaîne permettant d'insérer
#          directement des expressions Python.
# m.__name__ : récupère le nom de la bibliothèque.
# :<11 : aligne le nom à gauche dans un espace
#        d'une largeur minimale de 11 caractères.
#        Les espaces ajoutés facilitent l'alignement.
# m.__version__ : récupère le numéro de version
#                 installé pour chaque bibliothèque.
# print() : affiche le nom formaté suivi de sa version.
```



Task2—NumPy: vectorization, broadcasting, axes :

Rappel :

- **Vectorisation :** opération qui multiplie automatiquement chaque élément du tableau NumPy a par un nombre donné, sans utiliser de boucle for explicite en Python
- **Broadcasting :** mécanisme de NumPy permettant d'effectuer des opérations entre des tableaux et des valeurs de formes différentes, lorsque leurs dimensions sont compatibles. Scalar (scalaire) : une valeur numérique unique « nombre donné » : ajoute automatiquement sans utiliser de boucle for explicite.
- **Slicing (découpage) :** technique permettant d'extraire une partie des éléments d'un tableau NumPy.
  Syntaxe : a[début:fin:pas] (1 par défaut).
- **Negative indexing :** permet d'accéder aux éléments en comptant depuis la fin du tableau.
- **Boolean mask (masque booléen)** : c'est une technique de filtrage permettant de sélectionner les éléments d'un tableau NumPy qui respectent une condition logique.

  Cette comparaison génère un tableau de booléens : (True / False)

1) Vectorization

```
a = np.array([1, 2, 3, 4, 5])
print(a * 2)          # vectorized: no Python loop

✓ 0.0s
[ 2  4  6  8 10]
```



⇨ Elle est particulièrement utile en Machine Learning pour effectuer des calculs sur les données.

⇨ Les opérations NumPy sont généralement plus rapides, Le code devient plus court et plus lisible

⚠ Attention à la différence entre une liste Python et un tableau NumPy :

[1, 2, 3] * 2

⇨ Résultat : [1, 2, 3, 1, 2, 3] => La liste est répétée deux fois.

np.array([1, 2, 3]) * 2

⇨ Résultat : [2, 4, 6] => Chaque élément du tableau est multiplié par 2.

⚠ En NumPy, **un tableau unidimensionnel (1D)** peut être appelé **un vecteur**.

import numpy as np

a = np.array([1, 2, 3, 4, 5])
Vecteur 1D : shape = (5,)
⇨ *Un seul axe, sans distinction ligne/colonne*

b = np.array([[1, 2, 3, 4, 5]])
Vecteur ligne 2D : shape = (1, 5)
=> *Une ligne et cinq colonnes*

c = np.array([[1], [2], [3], [4], [5]])
Vecteur colonne 2D : shape = (5, 1)
⇨ *Cinq lignes et une colonne.*

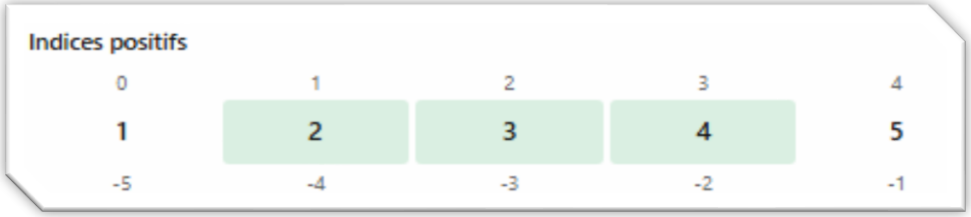
2) Broadcasting

```
a = np.array([1, 2, 3, 4, 5])
print(a + 10)          # broadcasting a scalar
```
0.0s
```
[11 12 13 14 15]
```

Mécanisme permettant à NumPy de réaliser des opérations entre des tableaux de formes compatibles, notamment entre un tableau et un scalaire

3) Slicing et Negative Indexing

```
a = np.array([1, 2, 3, 4, 5])
print(a[1:4], a[-1])  # slicing and negative indexing
```
0.0s
```
[2 3 4] 5
```



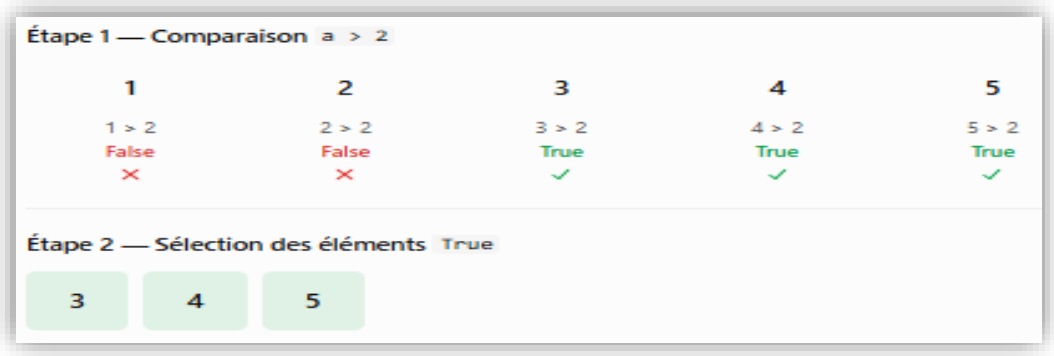
À retenir

- Slicing : permet d'extraire plusieurs éléments d'un tableau.
- Negative indexing : permet d'accéder aux éléments à partir de la fin avec des indices négatifs.
- Important : en Python, les indices commencent à 0 et la borne de fin d'un slicing est exclue.

4) Boolean Mask:

le masque booléen permet de filtrer les données selon une condition, sans écrire de boucle for.

```
a = np.array([1, 2, 3, 4, 5])
print(a[a > 2])          # boolean mask
```
0.0s
```
[3 4 5]
```



⚠ Différence entre les notions :

Notion	Définition
Boolean	Type de donnée logique : True ou False.
Boolean mask	Tableau de booléens indiquant les éléments à sélectionner.
Boolean indexing	Utilisation du masque booléen pour filtrer un tableau.

5) Axes : 2.2 the shape-predictiontable :

Dans une matrice 2D, axis=0 correspond à l'axe des lignes et axis=1 à celui des colonnes. Lors d'une réduction comme sum(), l'axe indiqué est celui qui est réduit : axis=0 produit une somme par colonne, tandis que axis=1 produit une somme par ligne.

Notion	Définition
Dimension (`ndim`)	Nombre d'axes d'un tableau NumPy.
Shape	Forme du tableau : nombre d'éléments sur chaque axe.
Axis	Axe selon lequel une opération est effectuée.
axis=0	Pour une matrice 2D, effectue une opération le long des lignes, donnant un résultat par colonne.
axis=1	Pour une matrice 2D, effectue une opération le long des colonnes, donnant un résultat par ligne.

```
import numpy as np
```

Importe la bibliothèque NumPy sous l'alias np.

```
M = np.arange(12).reshape(3, 4)
```

np.arange(12) : génère 12 entiers de 0 à 11.

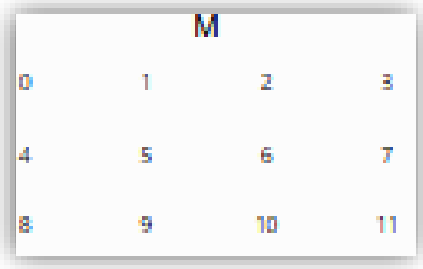
reshape(3, 4) : transforme le tableau 1D en matrice 2D.

3 : nombre de lignes.

4 : nombre de colonnes.

M.shape = (3, 4)

M.ndim = 2




```
v = np.array([10, 20, 30, 40])                    # shape (4,)
```

Crée un vecteur NumPy unidimensionnel de 4 éléments.

v.shape = (4,).

Ce vecteur peut être diffusé (broadcasting) sur chaque ligne de M.




```
w = np.array([[100], [200], [300]])               # shape (3, 1)
```

Crée un tableau NumPy bidimensionnel.

w contient 3 lignes et 1 colonne.

w.shape = (3, 1).

Ce vecteur colonne peut être diffusé sur chaque colonne de M.

[[100]

[200]

[300]]

print(M)

Affiche la matrice M de dimension 3 × 4.

[0 1 2 3]

[4 5 6 7]

[8 9 10 11]

TODO: For each expression below, print the result AND its shape

M.mean(axis=0), M.mean(axis=1), M + V, M + W

print(M.mean(axis=0), M.mean(axis=0).shape)

mean() : calcule la moyenne arithmétique.

axis=0 : calcule une moyenne pour chaque colonne.

C'est-à-dire :

Colonne 1 : (0 + 4 + 8) / 3 = **4**

Colonne 2 : (1 + 5 + 9) / 3 = **5**

Colonne 3 : (2 + 6 + 10) / 3 = **6**

Colonne 4 : (3 + 7 + 11) / 3 = **7**

Résultat : [4. 5. 6. 7.]

Shape : (4,), car il existe **4 colonnes.**

print(M.mean(axis=1), M.mean(axis=1).shape)

axis=1 : calcule une moyenne pour chaque ligne.

Ligne 1 : (0 + 1 + 2 + 3) / 4 = **1.5**

Ligne 2 : (4 + 5 + 6 + 7) / 4 = **5.5**

Ligne 3 : (8 + 9 + 10 + 11) / 4 = **9.5**

Résultat : [1.5 5.5 9.5]

Shape : (3,), car il existe **3 lignes.**

print(M + v, (M + v).shape)

Broadcasting : NumPy additionne le vecteur v à chacune des 3 lignes de la matrice M.

C'est-à-dire :

M.shape = (3, 4).

v.shape = (4,).

Les dimensions sont compatibles.

Résultat :

[10 21 32 43]

[14 25 36 47]

[18 29 40 51]

Shape : (3, 4).

print(M + w, (M + w).shape)

Broadcasting : NumPy additionne chaque élément de w à tous les éléments de la ligne correspondante de M.

C’est-à-dire :

M.shape = (3, 4).

w.shape = (3, 1).

Les dimensions sont compatibles.

Résultat :

[100 101 102 103]

[204 205 206 207]

[308 309 310 311]

Shape : (3, 4)

try:

* try : permet de tester une instruction susceptible de provoquer une erreur pendant son exécution.

```
except ValueError as e:
    print("Erreur :", e)
```

except ValueError as e:

C’est-à-dire :

except : intercepte l'erreur ValueError.

ValueError : erreur liée à une valeur ou une forme incompatible avec l'opération demandée.

as e : stocke l'exception dans la variable e.

```
print(M + np.array([1, 2, 3]))
```

Crée un vecteur de shape (3,).

Tente de l'additionner à M de shape (3, 4).

Broadcasting impossible :

Les dimensions finales 4 et 3 sont incompatibles.

Une erreur ValueError est déclenchée.

print("Erreur :", e)

⇨ Affiche le message d'erreur sans arrêter le programme.

```python
import numpy as np

M = np.arange(12).reshape(3, 4)         # 3 rows, 4 columns

v = np.array([10, 20, 30, 40])          # shape (4,)


w = np.array([[100], [200], [300]])     # shape (3, 1)


print(M)

#TODO

print(M.mean(axis=0), M.mean(axis=0).shape)


print(M.mean(axis=1), M.mean(axis=1).shape)


print(M + v, (M + v).shape)


print(M + w, (M + w).shape)


try:
    print(M + np.array([1, 2, 3]))


except ValueError as e:
      print("Erreur :", e)
```

2.3. Standardize columns without a loop :

1. Objectif de l'exercice

L'objectif est de standardiser chaque colonne d'une matrice NumPy, en utilisant la vectorisation et le broadcasting, sans boucle for.

La matrice X_small contient des données synthétiques représentant quatre personnes :

Personne	Taille (cm)	Masse (kg)
1	150	50
2	160	60
3	170	80
4	180	90

⇨ La standardisation (ou Z-score normalization) transforme les données afin que chaque colonne ait une moyenne égale à 0 et un écart-type égal à 1.

$$Z_{ij} = \frac{X_{ij} - \mu_j}{\sigma_j}$$

Où :

- X_{ij} : valeur de la ligne i, colonne j.
- μ_j : moyenne de la colonne j.
- σ_j : écart-type de la colonne j.
- Z_{ij} : valeur standardisée.

2. Code Python complet avec commentaires

```
import numpy as np

# Importe la bibliothèque NumPy sous l'alias np.

X_small = np.array([[150., 50.],
                    [160., 60.],
                    [170., 80.],
                    [180., 90.]])

# Crée une matrice NumPy bidimensionnelle de shape (4, 2).

# Chaque ligne représente une personne.

# Colonne 0 : taille en centimètres.

# Colonne 1 : masse en kilogrammes.

# Le point décimal indique des valeurs de type float.

Z = (X_small - X_small.mean(axis=0)) / X_small.std(axis=0)

# Standardise toutes les colonnes en une seule expression vectorisée.

# X_small.mean(axis=0) : calcule la moyenne de chaque colonne.

# Résultat : [165. 70.]

# X_small.std(axis=0) : calcule l'écart-type de chaque colonne.

# Par défaut, NumPy utilise ddof=0 (écart-type de population).

# Résultat approximatif : [11.180 15.811].

# X_small - moyenne : centre les données autour de zéro.

# La division par l'écart-type normalise leur dispersion.

# Broadcasting : NumPy applique les deux vecteurs de shape (2,)

# à toutes les lignes de X_small, de shape (4, 2).

# Aucune boucle for explicite n'est nécessaire.

# Z conserve la shape (4, 2).

print("Z =", Z)

# Affiche la matrice standardisée.

# Chaque valeur indique son écart à la moyenne

# exprimé en nombre d'écarts-types.

print("Moyennes :", Z.mean(axis=0))

# Calcule la moyenne de chaque colonne de Z.

# Résultat attendu : [0. 0.] approximativement.

# De très petites erreurs d'arrondi peuvent apparaître.

print("Écarts-types :", Z.std(axis=0))

# Calcule l'écart-type de chaque colonne standardisée.

# Résultat attendu : [1. 1.] approximativement.
```

```
print("Vérification :", np.allclose(Z.mean(axis=0), 0))

# np.allclose() : vérifie l'égalité approximative

# entre les moyennes calculées et zéro.

# Résultat attendu : True.

print("Vérification :", np.allclose(Z.std(axis=0), 1))

# Vérifie que les écarts-types des colonnes

# sont approximativement égaux à 1.

# Résultat attendu : True.
```

```
Z =
 [[-1.34164079 -1.34164079]
 [-0.4472136  -0.4472136 ]
 [ 0.4472136   0.4472136 ]
 [ 1.34164079  1.34164079]]
Moyennes de Z : [0. 0.]
Écarts-types de Z : [1. 1.]
```

Les formes des tableaux

X_small
Shape (4, 2)

150	50
160	60
170	80
180	90

Moyennes
Shape (2,)

165	70

Répétées sur les 4 lignes

Colonne 0 — Taille

$$\mu_0 = \frac{150 + 160 + 170 + 180}{4} = \boxed{165}$$

Colonne 1 — Masse

$$\mu_1 = \frac{50 + 60 + 80 + 90}{4} = \boxed{70}$$

⇨ NumPy soustrait automatiquement 165 à chaque taille et 70 à chaque masse. Ensuite, chaque colonne est divisée par son propre écart-type.

3. Pourquoi standardiser les données en Machine Learning ?

Les caractéristiques peuvent avoir des unités et des échelles différentes (centimètres, kilogrammes, euros, etc.). La standardisation limite l'influence de ces différences d'échelle sur certains algorithmes, notamment kNN, SVM, K-Means et les modèles utilisant la descente de gradient.

⚠Attention : dans cet exercice, on standardise directement X_small pour apprendre le calcul. Dans un véritable pipeline de Machine Learning, on calcule la moyenne et l'écart-type uniquement sur les données d'entraînement, puis on utilise ces mêmes paramètres pour transformer les ensembles de validation et de test, afin d'éviter la fuite de données (data leakage).

Q2.

a. Why does `M + np.array([1, 2, 3])` fail while `M + w` works? State the broadcasting rule you used.

Answer ; a:

- M.shape = (3, 4)
- np.array([1, 2, 3]).shape = (3,)
- w.shape = (3, 1)

⇨ L'opération M + np.array([1, 2, 3]) échoue parce que les dimensions des deux tableaux sont incompatibles pour le broadcasting.

Règle du broadcasting : NumPy compare les dimensions de droite à gauche. Deux dimensions sont compatibles si elles sont égales ou si l'une d'elles vaut 1. Les dimensions manquantes sont considérées comme égales à 1.

Opération	Comparaison des shapes	Résultat
M + np.array([1,2,3])	(3,4) et (1,3)	Incompatible : 4 ≠ 3
M + w	(3,4) et (3,1)	Compatible : 3 = 3 et 1 peut être diffusé en 4

b. In one sentence: what does **vectorized** mean and why do we prefer it to Python loops?

Answer ; b:

Une opération vectorisée applique directement un calcul aux éléments d'un tableau NumPy sans boucle Python explicite, ce qui rend le code plus simple, plus lisible et généralement plus rapide.

Task3— Exploringthe data withPandas

L'objectif est de manipuler un DataFrame Pandas, d'explorer sa structure, d'ajouter une colonne contenant les noms des espèces et de compter le nombre d'échantillons de chaque espèce.

```
iris = load_iris(as_frame=True)
```

load_iris() : charge le dataset Iris depuis scikit-learn. as_frame=True : retourne les données sous forme de DataFrame Pandas. Le dataset contient 150 fleurs appartenant à 3 espèces. Chaque fleur possède 4 caractéristiques numériques :

- Longueur du sépale (sepal length).

- Largeur du sépale (sepal width).

- Longueur du pétale (petal length).

- Largeur du pétale (petal width).

```
df = iris.frame.copy()
```

iris.frame : DataFrame contenant les 4 caractéristiques et une colonne target (classe de la fleur).

```
# copy() : crée une copie indépendante du DataFrame.

# df : variable contenant cette copie.

# Shape initiale : (150, 5).

features = iris.feature_names

# Récupère les noms des 4 caractéristiques du dataset. Les noms sont stockés dans une liste Python. Cette liste pourra être utilisée pour sélectionner les colonnes des caractéristiques.

# -------- TODO 3.1 : Explorer le DataFrame --------

print(df.head())

# head() : affiche les 5 premières lignes par défaut.

# Permet d'observer les valeurs et les noms des colonnes.

print(df.shape)

# shape : retourne les dimensions du DataFrame.

# Résultat : (150, 5).

# 150 : nombre de lignes (échantillons).

# 5 : nombre de colonnes (4 features + target).

df.info()

# info() : affiche un résumé de la structure du DataFrame.

# Indique les noms des colonnes, les types de données,

# le nombre de valeurs non nulles et l'utilisation mémoire.

# Cette méthode affiche directement le résultat.

# Il n'est donc pas nécessaire d'utiliser print().

# -------- TODO 3.2 : Ajouter la colonne species --------

species_dict = {

    0: "setosa",

    1: "versicolor",

    2: "virginica"

}

# Crée un dictionnaire Python associant

# chaque numéro de classe au nom de son espèce.

# 0 correspond à setosa.

# 1 correspond à versicolor.

# 2 correspond à virginica.

df["species"] = df["target"].map(species_dict)

# df["target"] : sélectionne la colonne des classes numériques.

# map() : remplace chaque valeur par le nom correspondant

# dans le dictionnaire species_dict.
```

```
# df["species"] : crée une nouvelle colonne appelée species.

# Le DataFrame contient maintenant 6 colonnes.

# Nouvelle shape : (150, 6).

print(df.head())

# Affiche les cinq premières lignes avec la nouvelle

# colonne species.

# -------- TODO 3.3 : Compter les espèces --------

print(df["species"].value_counts())

# df["species"] : sélectionne la colonne des espèces.

# value_counts() : compte le nombre d'occurrences

# de chaque valeur distincte dans cette colonne.

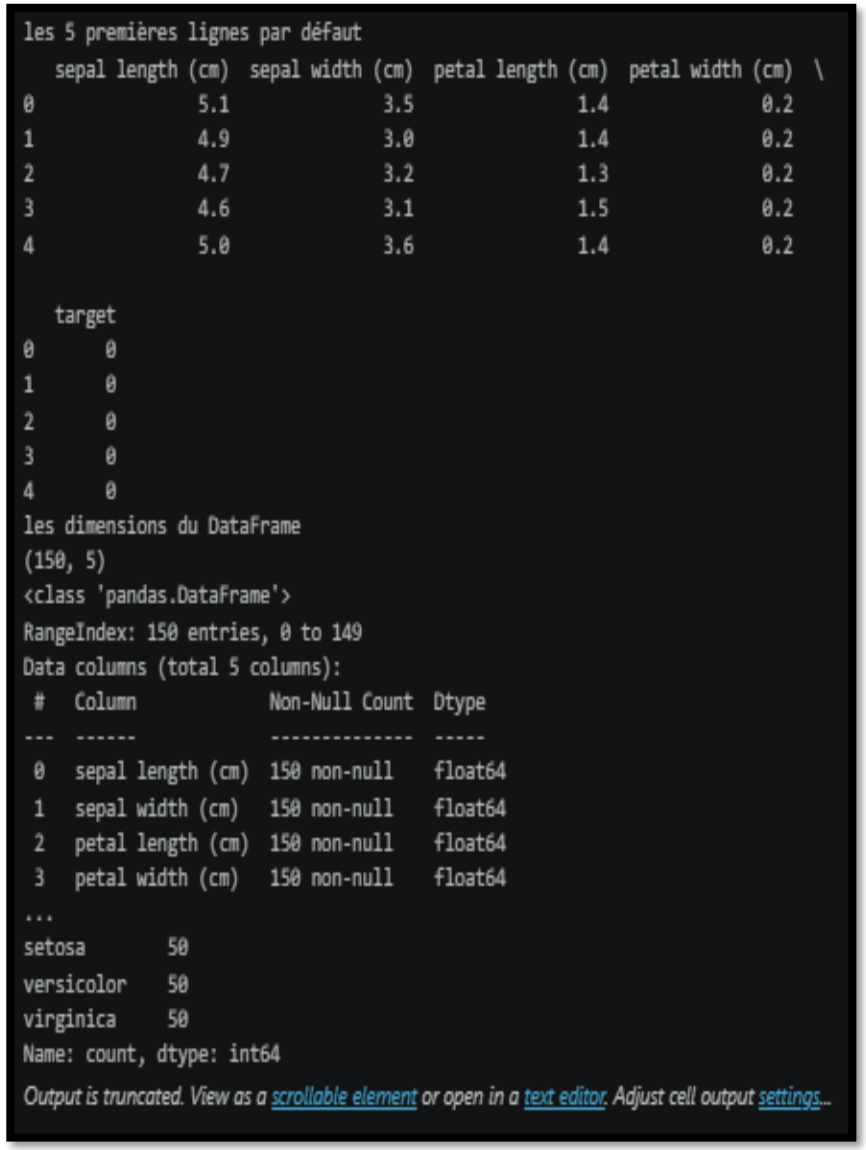
# Résultat attendu :

# setosa       50

# versicolor   50

# virginica    50

# Cela montre que le dataset est équilibré.
```



```
les 5 premières lignes par défaut
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   target
0       0
1       0
2       0
3       0
4       0
les dimensions du DataFrame
(150, 5)
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
...
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

- Cette instruction traduit automatiquement les étiquettes numériques en noms compréhensibles. Elle conserve également la colonne `target`, qui pourra servir à l'entraînement d'un modèle de classification.

- Conclusion : le dataset Iris contient 150 observations, 4 caractéristiques numériques et 3 classes équilibrées de 50 observations chacune. Après l'ajout de la colonne `species`, le DataFrame possède 6 colonnes.

TODO 3.4 — Global summary statistics of the 4 features (`describe()`)

Objectif : afficher les statistiques descriptives des quatre caractéristiques du dataset Iris : moyenne, écart-type, minimum, maximum et quartiles.

```
print(df[features].describe())
```

df : DataFrame Pandas contenant les données Iris.

features : liste des noms des 4 caractéristiques numériques.

df[features] : sélectionne uniquement ces 4 colonnes.

describe() : calcule les statistiques descriptives de chaque caractéristique sélectionnée.

count : nombre de valeurs non nulles.

mean : moyenne arithmétique.

std : écart-type des valeurs (ddof=1 dans Pandas).

min : valeur minimale.

25% : premier quartile (Q1).

50% : médiane (Q2).

75% : troisième quartile (Q3).

max : valeur maximale.

print() : affiche le tableau des statistiques.

```
       sepal length (cm)  sepal width (cm)  petal length (cm)
count         150.000000        150.000000         150.000000
mean            5.843333          3.057333           3.758000
std             0.828066          0.435866           1.765298
min             4.300000          2.000000           1.000000
25%             5.100000          2.800000           1.600000
50%             5.800000          3.000000           4.350000
75%             6.400000          3.300000           5.100000
max             7.900000          4.400000           6.900000

       petal width (cm)
count        150.000000
mean           1.199333
std            0.762238
min            0.100000
25%            0.300000
50%            1.300000
75%            1.800000
max            2.500000
```

TODO 3.5 — Mean of each feature per species (`groupby()`)

Objectif : calculer la moyenne de chaque caractéristique séparément pour les trois espèces : Setosa, Versicolor et Virginica.

```
print(df.groupby("species")[features].mean())
```

groupby("species") : regroupe les observations selon la valeur de la colonne species. On obtient 3 groupes :

- setosa (50 observations).

- versicolor (50 observations).

- virginica (50 observations).

[features] : sélectionne les 4 caractéristiques numériques dans chaque groupe.

mean() : calcule la moyenne de chaque caractéristique pour chaque espèce séparément.

print() : affiche le tableau des moyennes.

Résultat : DataFrame de shape (3, 4).

3 lignes : une par espèce.

4 colonnes : une par caractéristique.

```
            sepal length (cm)  sepal width (cm)  petal length (cm)
species
setosa                  5.006             3.428              1.462
versicolor              5.936             2.770              4.260
virginica               6.588             2.974              5.552

            petal width (cm)
species
setosa                 0.246
versicolor             1.326
virginica              2.026
```

Comprendre la différence entre `describe()` et `groupby()`

describe()	groupby("species").mean()
Statistiques sur l'ensemble des 150 fleurs.	Statistiques calculées séparément pour chaque espèce.
Fournit plusieurs indicateurs : mean, std, min, quartiles, max.	Fournit ici uniquement la moyenne de chaque caractéristique.
Permet d'analyser la distribution globale.	Permet de comparer les trois espèces.

Q3 — Correction avec explications

Question : Looking at the per-species means and the global standard deviations, which feature seems to change the most from one species to another relative to its spread? Which seems least useful? Quote numbers.

1. Comprendre la question

On cherche à identifier :

- La caractéristique la plus discriminante : celle dont les moyennes varient fortement entre les espèces, relativement à son écart-type global.
- La caractéristique la moins discriminante : celle dont les moyennes varient le moins, relativement à son écart-type global.

Pour comparer les caractéristiques, on peut utiliser le rapport :

$$R = \frac{\max(\text{moyennes}) - \min(\text{moyennes})}{\text{écart-type global}}$$

Comparaison numérique

Feature	Min. moyenne	Max. moyenne	Écart-type global	Rapport R
Sepal length	5.006	6.588	0.828	1.91
Sepal width	2.770	3.428	0.436	1.51
Petal length	1.462	5.552	1.765	2.32
Petal width	0.246	2.026	0.762	2.34

Interprétation

Caractéristique la plus discriminante : Petal width (largeur des pétales).

Les moyennes passent de 0.246 cm pour Setosa à 2.026 cm pour Virginica, soit une différence de 1.780 cm. Rapportée à l'écart-type global de 0.762 cm, cette différence représente environ 2.34 écarts-types.

La longueur des pétales (Petal length) est également très discriminante, avec un rapport de 2.32, très proche.

Caractéristique la moins discriminante : Sepal width (largeur des sépales).

Les moyennes varient de 2.770 à 3.428 cm, soit une différence de 0.658 cm. Rapportée à l'écart-type global de 0.436 cm, elle représente environ 1.51 écart-type, le rapport le plus faible des quatre caractéristiques.

- **Réponse en anglais à écrire dans le TP**

Petal width appears to be the most discriminative feature, with species means ranging from 0.246 to 2.026 cm and a global standard deviation of 0.762 cm (ratio ≈ 2.34). Sepal width seems the least useful, with means ranging from 2.770 to 3.428 cm and a standard deviation of 0.436 cm (ratio ≈ 1.51).

Remarque scientifique : cette conclusion est exploratoire. Comparer les moyennes et les écarts-types globaux donne une première indication, mais ne suffit pas à mesurer précisément le pouvoir discriminant d'une caractéristique. Il faudrait également examiner les distributions au sein de chaque espèce et leurs chevauchements.

Task4—Seeingthe structure withMatplotlib

Matplotlib : Visualisation des données Iris

L'objectif est de créer une figure contenant trois graphiques côte à côte pour comparer les caractéristiques des trois espèces Iris : setosa, versicolor et virginica.

1. Code complet avec commentaires sous chaque instruction

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# plt.subplots() : crée une figure contenant plusieurs graphiques.

# 1 : nombre de lignes de graphiques.

# 3 : nombre de colonnes de graphiques.

# figsize=(15, 4) : définit la taille de la figure en pouces.

# fig : représente la figure complète.

# axes : tableau contenant les trois zones de dessin.

# axes[0] : premier graphique.
```

```
# axes[1] : deuxième graphique. # axes[2] : troisième graphique.
# -------- TODO 4.1 : Petal length vs Petal width --------
for name, group in df.groupby("species"):
    # groupby("species") : regroupe les fleurs selon leur espèce.
    # La boucle parcourt les trois groupes.
    # name : nom de l'espèce (setosa, versicolor, virginica).
    # group : DataFrame contenant les fleurs de cette espèce.
    axes[0].scatter(
        group["petal length (cm)"],
        group["petal width (cm)"],
        label=name
    )
    # axes[0] : sélectionne le premier graphique.
    # scatter() : crée un nuage de points.
    # Premier argument : valeurs de l'axe horizontal X.
    # Ici, la longueur des pétales.
    # Deuxième argument : valeurs de l'axe vertical Y.
    # Ici, la largeur des pétales.
    # label=name : associe le nom de l'espèce aux points.
    # Matplotlib attribue automatiquement une couleur
    # différente à chaque espèce.
axes[0].set_xlabel("Petal length (cm)")
# Définit le titre de l'axe horizontal du premier graphique.
axes[0].set_ylabel("Petal width (cm)")
# Définit le titre de l'axe vertical du premier graphique.
axes[0].set_title("Petal length vs Petal width")
# Ajoute un titre au premier graphique.
axes[0].legend()
# Affiche la légende associant chaque couleur à une espèce.
# -------- TODO 4.2 : Sepal length vs Sepal width --------
for name, group in df.groupby("species"):
    # Parcourt les trois groupes d'espèces Iris.
    axes[1].scatter(
        group["sepal length (cm)"],
        group["sepal width (cm)"],
        label=name
```

```
    )
    # axes[1] : sélectionne le deuxième graphique.
    # scatter() : crée un nuage de points.
    # Axe X : longueur des sépales.
    # Axe Y : largeur des sépales.
    # label=name : identifie l'espèce dans la légende.
axes[1].set_xlabel("Sepal length (cm)")
# Définit le titre de l'axe horizontal.
axes[1].set_ylabel("Sepal width (cm)")
# Définit le titre de l'axe vertical.
axes[1].set_title("Sepal length vs Sepal width")
# Définit le titre du deuxième graphique.
axes[1].legend()
# Affiche la légende des espèces.
# -------- TODO 4.3 : Petal length histograms --------
for name, group in df.groupby("species"):
    # Parcourt les trois espèces séparément.
    axes[2].hist(
        group["petal length (cm)"],
        bins=10,
        alpha=0.5,
        label=name
    )
    # axes[2] : sélectionne le troisième graphique.
    # hist() : crée un histogramme.
    # group["petal length (cm)"] : données à représenter.
    # bins=10 : divise les valeurs en 10 intervalles.
    # alpha=0.5 : définit la transparence à 50 %.
    # label=name : nom de l'espèce dans la légende.
    # Les histogrammes sont superposés pour comparer
    # les distributions des trois espèces.
axes[2].set_xlabel("Petal length (cm)")
# Définit le titre de l'axe horizontal.
axes[2].set_ylabel("Frequency")
# Axe Y : nombre d'observations par intervalle.
axes[2].set_title("Petal length distribution")
```

Définit le titre du troisième graphique.

axes[2].legend()

Affiche la légende des histogrammes.

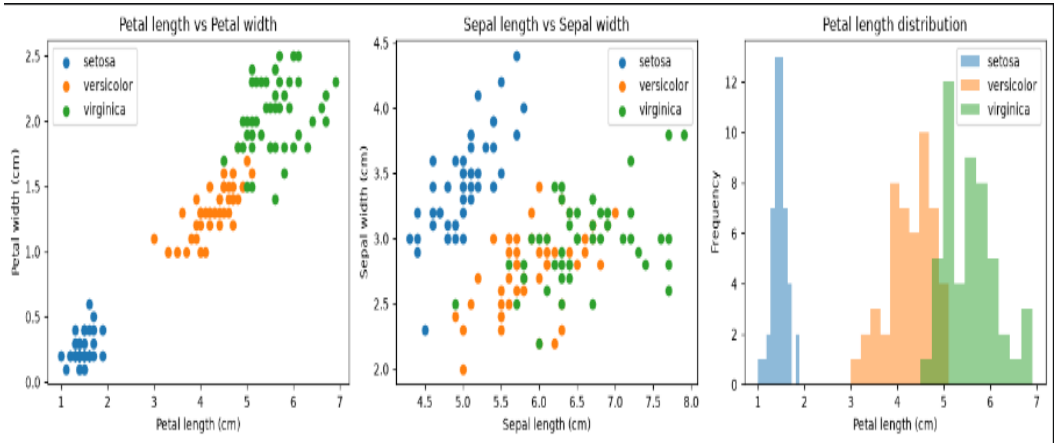
plt.tight_layout()

Ajuste automatiquement les espacements entre graphiques.

Évite les chevauchements des titres et des étiquettes.

plt.show()

Affiche la figure complète contenant les trois graphiques.



Premier graphique — Petal length vs Petal width : les trois espèces forment des groupes relativement distincts. Setosa est particulièrement facile à séparer. Versicolor et Virginica présentent un chevauchement limité.

Deuxième graphique — Sepal length vs Sepal width : les points des trois espèces se chevauchent davantage, particulièrement ceux de Versicolor et Virginica. Ces deux caractéristiques sont donc moins efficaces pour distinguer visuellement les espèces.

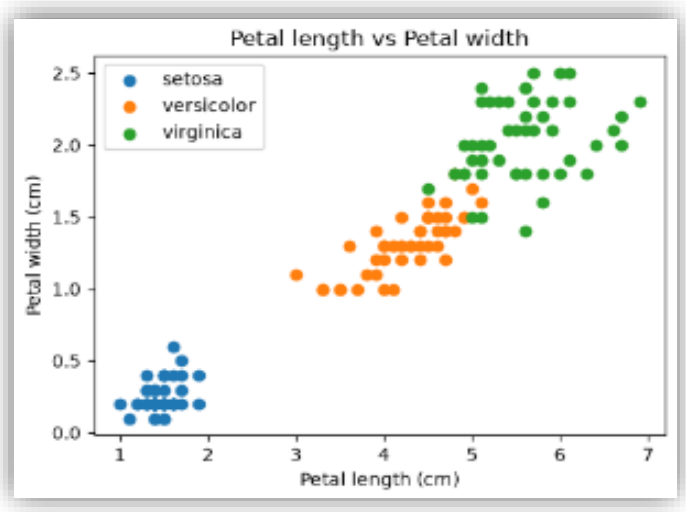
Troisième graphique — Petal length histogram : la distribution de Setosa est clairement séparée des deux autres. Versicolor et Virginica montrent un certain chevauchement de leurs longueurs de pétales.

➢ Fonctions Matplotlib à retenir

Fonction	Définition
plt.subplots()	Crée une figure avec plusieurs graphiques
scatter()	Représente des données sous forme de points
hist()	Affiche la distribution des valeurs par intervalles
set_xlabel()	Définit l'étiquette de l'axe X
set_ylabel()	Définit l'étiquette de l'axe Y
set_title()	Définit le titre du graphique
legend()	Affiche la légende
bins	Définit les intervalles d'un histogramme
alpha	Contrôle la transparence des éléments
tight_layout()	Ajuste l'espacement des graphiques
show()	Affiche la figure

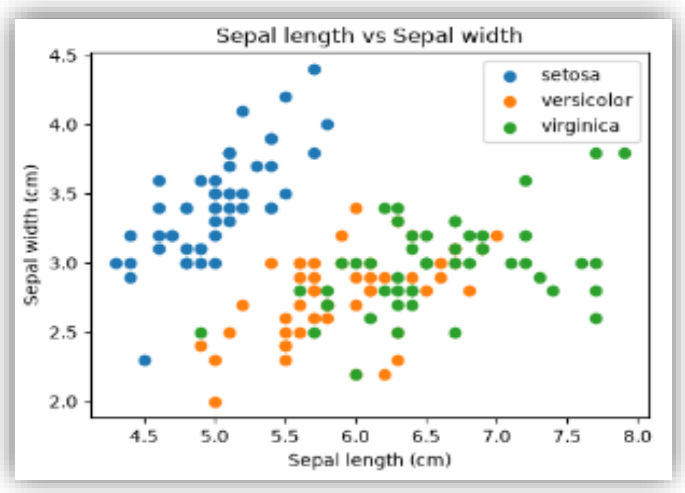
Conclusion pour le TP : les caractéristiques des pétales permettent de mieux distinguer les espèces Iris que celles des sépales. Cela rejoint l'analyse statistique réalisée dans la Task 3.

Q4. préciser explicitement où le classificateur risque de commettre des erreurs ?



- Petal length vs Petal width :

Setosa est clairement séparée des deux autres espèces. Versicolor et Virginica présentent un léger chevauchement



- Sepal length vs Sepal width :

les trois espèces se chevauchent davantage, notamment Versicolor et Virginica.

- **Petal length histogram :** Setosa possède des pétales courts, tandis que les distributions de Versicolor et Virginica se chevauchent partiellement.

⇨ La prédiction des erreurs repose sur le chevauchement des classes (class overlap).

Confusion Matrix — Matrice de confusion

Définition : la matrice de confusion est un tableau permettant d'évaluer les performances d'un modèle de classification en comparant les classes réelles (Actual) aux classes prédites (Predicted)

Matrice de confusion illustrative

Colonnes : classes prédites (Predicted)

Réel ↓	Setosa	Versicolor	Virginica
Setosa	10	0	0
Versicolor	0	8	2
Virginica	0	1	9

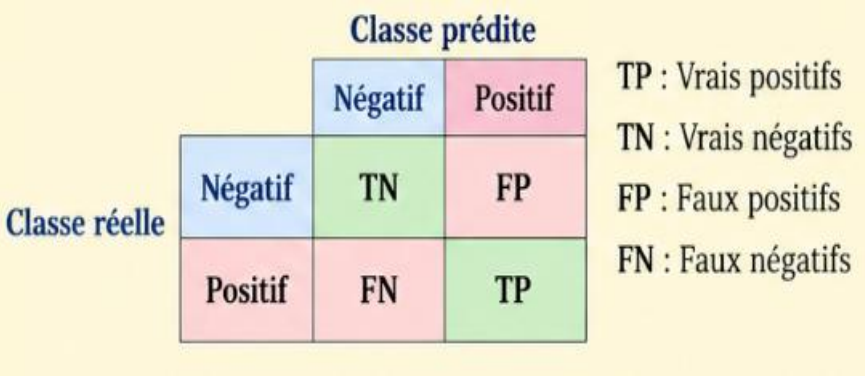
- Diagonale principale (10, 8, 9) : prédictions correctes.
- Hors diagonale : erreurs de classification.
- 2 Versicolor ont été prédites comme Virginica.
- 1 Virginica a été prédite comme Versicolor.

A retenir :

Matrice de confusion (Confusion Matrix) : tableau utilisé pour évaluer un modèle de classification en comparant les classes réelles aux classes prédites, afin d'identifier les prédictions correctes et les erreurs de classification.

À retenir :

- Diagonale principale : prédictions correctes.
- Hors diagonale : erreurs de classification.
- Lignes : classes réelles (Actual).
- Colonnes : classes prédites (Predicted).



Task5—A first honest model

Task 5.1 — Features, target and a stratified split

Question 1 — Préparer X et y

```
X = df[features].to_numpy()
y = df["target"].to_numpy()
```
✓ 0.0s

X = df[features].to_numpy()

df : DataFrame contenant les 150 fleurs Iris.

features : liste des noms des 4 caractéristiques.

df[features] : sélectionne les 4 colonnes des caractéristiques.

to_numpy() : convertit les données Pandas en tableau NumPy.

X : matrice des caractéristiques utilisée par le modèle.

X.shape = (150, 4).

y = df["target"].to_numpy()

df["target"] : sélectionne la colonne des classes.

to_numpy() : convertit cette colonne en tableau NumPy.

y : vecteur contenant les classes réelles.

0 = Setosa, 1 = Versicolor, 2 = Virginica.

y.shape = (150,).

le Stratified Split

Consigne de TP: Split with test_size=0.3, random_state=SEED, stratify=y into X_train, X_test, y_train, y_test.

⇨ Il faut diviser les 150 observations en deux ensembles.




```
SEED = 42

# Fixe une graine aléatoire pour la reproductibilité.

X_train, X_test, y_train, y_test = train_test_split(

   X, y, test_size=0.3, random_state=SEED, stratify=y

)

# train_test_split() : divise le dataset en Train et Test ; X, y : caractéristiques et classes à répartir.

# test_size=0.3 : réserve 30 % des données pour Test.

# random_state=SEED : rend le découpage reproductible.

# stratify=y : conserve les proportions des espèces.

# X_train : caractéristiques des fleurs d'entraînement.

# X_test : caractéristiques des fleurs de test.

# y_train : vraies classes des fleurs d'entraînement.

# y_test : vraies classes des fleurs de test.
```

Afficher les shapes et les class counts

Consigne de exercise: Print the shapes and the class counts in each part using `np.bincount`.

```
print("X_train :", X_train.shape)

# Affiche (105, 4).

# 105 observations et 4 caractéristiques.

print("X_test :", X_test.shape)

# Affiche (45, 4).

# 45 observations et 4 caractéristiques.

print("y_train :", y_train.shape)

# Affiche (105,).

# 105 étiquettes de classes.

print("y_test :", y_test.shape)

# Affiche (45,).
```

45 étiquettes de classes.

print("Train class counts :", np.bincount(y_train))

np.bincount() : compte les occurrences de chaque classe.

Résultat : [35 35 35].

print("Test class counts :", np.bincount(y_test))

Compte les observations de chaque classe dans Test.

Résultat : [15 15 15].

Espèce	Train	Test
Setosa	35	15
Versicolor	35	15
Virginica	35	15
Total	**105**	**45**

⇨ Cette répartition confirme que la stratification a conservé les proportions des trois espèces ; la stratification Pour conserver les proportions des classes dans Train et Test.

Task 5.2 — Baseline, then kNN

Question 1 — Majority-class baseline

Consigne du TP:

TODO: majority-class baseline: DummyClassifier(strategy="most_frequent"), fit on TRAIN, accuracy on TEST.

Objectif : créer un modèle de référence (baseline) qui prédit toujours la classe majoritaire dans Train, puis mesurer son accuracy sur Test.

baseline = DummyClassifier(strategy="most_frequent")

DummyClassifier : classe de scikit-learn permettant de créer un modèle de classification de référence.

strategy="most_frequent" : choisit la classe la plus fréquente dans l'ensemble d'entraînement.

baseline : variable contenant ce modèle.

baseline.fit(X_train, y_train)

fit() : ajuste le modèle sur les données Train.

X_train : caractéristiques des 105 fleurs.

y_train : classes réelles des 105 fleurs.

Le modèle détermine la classe majoritaire dans y_train.

Ici, les 3 classes possèdent chacune 35 observations.

En cas d'égalité, la plus petite étiquette est choisie.

Le modèle sélectionne donc la classe 0 : Setosa.

y_baseline = baseline.predict(X_test)

predict() : génère les prédictions du modèle.

X_test : caractéristiques des 45 fleurs de test.

La baseline prédit toujours Setosa (0).

y_baseline : tableau contenant 45 prédictions.

```
baseline_acc = accuracy_score(y_test, y_baseline)
```

accuracy_score() : calcule la proportion des prédictions correctes sur Test.

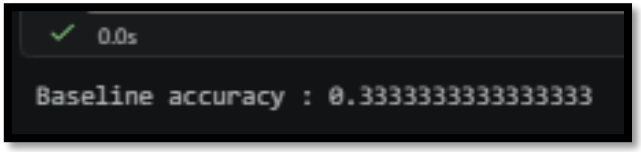
y_test : vraies classes des 45 fleurs.

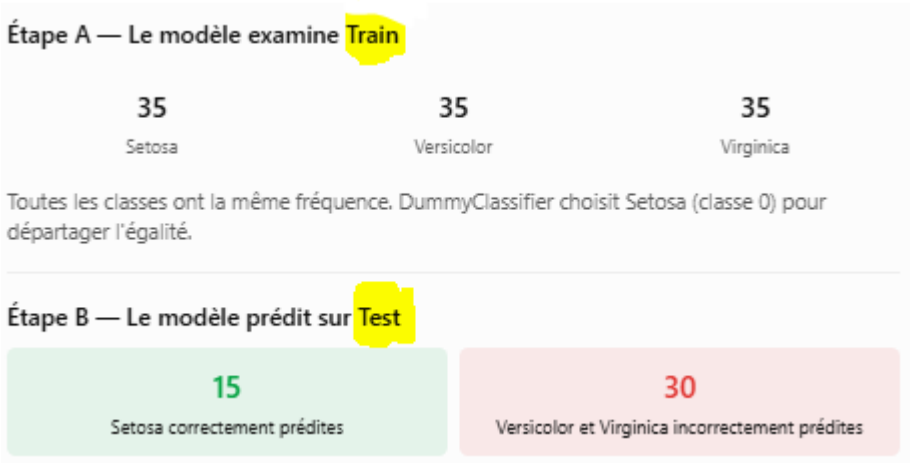
y_baseline : classes prédites par DummyClassifier.

15 prédictions correctes sur 45 (Résultat : 15 / 45 = 0.3333)

```
print("Baseline accuracy :", baseline_acc)
```

Affiche : Baseline accuracy : 0.3333333333333333.





Calcul de l'Accuracy :

$$Accuracy_{baseline} = \frac{15}{45} = 0,3333$$

Soit :

$$Accuracy_{baseline} = 33,33\%$$

Interprétation : notre modèle de référence classe correctement un tiers des fleurs de Test, sans utiliser les quatre caractéristiques pour distinguer les espèces.

Question 2: KNeighborsClassifier (kNN)

Consigne du TP:

TODO: KNeighborsClassifier(n_neighbors=3), fit on TRAIN, accuracy on TEST.

TODO: also print the kNN accuracy on TRAIN (for comparison only).

TODO: print the improvement over the baseline in percentage points.

1. Objectif

Nous allons entraîner un classificateur kNN (k-Nearest Neighbors), puis comparer ses performances avec celles de la baseline `DummyClassifier`.

Le modèle kNN utilise les caractéristiques des fleurs pour prédire leur espèce.

2. Définition de kNN

k-Nearest Neighbors (k plus proches voisins) est un algorithme de Machine Learning supervisé qui classe une nouvelle observation selon la classe majoritaire parmi ses `k` plus proches voisins de l'ensemble d'entraînement.

Dans notre TP, `k=3`.

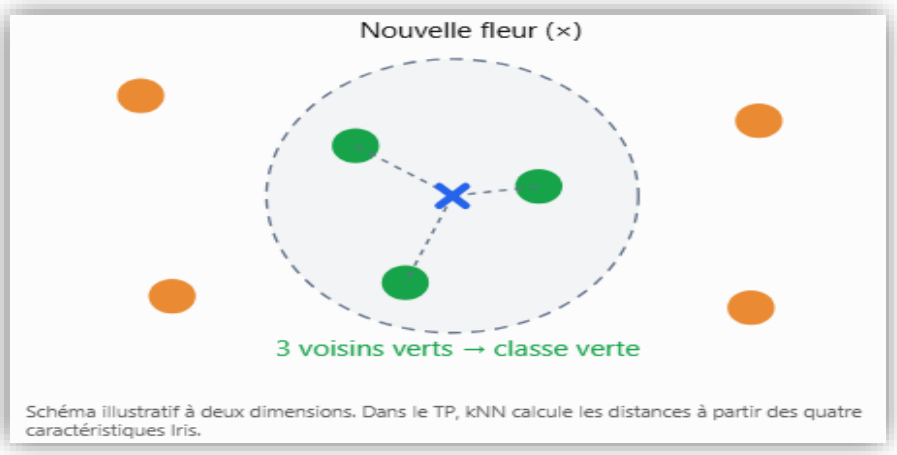


Schéma illustratif à deux dimensions. Dans le TP, kNN calcule les distances à partir des quatre caractéristiques Iris.

```
knn = KNeighborsClassifier(n_neighbors=3)

# KNeighborsClassifier : crée un modèle de classification kNN.

# n_neighbors=3 : fixe le nombre de voisins à examiner.

# Pour chaque nouvelle fleur, le modèle recherche les 3 observations les plus proches dans les données Train.

# La classe majoritaire parmi les 3 voisins est prédite. Par défaut, la distance utilisée est euclidienne.

knn.fit(X_train, y_train)

# fit() : ajuste le modèle sur les données d'entraînement.

# X_train : les 4 caractéristiques des 105 fleurs Train.

# y_train : les espèces réelles de ces 105 fleurs.

# kNN conserve les données d'entraînement pour rechercher les voisins lors des prédictions.

y_pred = knn.predict(X_test)

# predict() : prédit les espèces des 45 fleurs de Test.

# Pour chaque fleur, kNN identifie les 3 voisins les plus proches parmi les 105 fleurs Train.

# y_pred : tableau contenant les 45 classes prédites.
```

```
test_acc = accuracy_score(y_test, y_pred)

# Calcule l'accuracy du modèle sur Test, Compare les 45 classes réelles avec les 45 prédictions.

# Résultat attendu : environ 0.9556, soit 95.56%.

# 43 fleurs sont correctement classées sur 45.

train_acc = knn.score(X_train, y_train)

# Calcule l'accuracy sur les données Train.

# Résultat attendu : environ 0.9619, soit 96.19%.

# Sert uniquement à comparer Train et Test. Ce n'est pas la performance principale à rapporter.

improvement = (test_acc - baseline_acc) * 100

# Calcule l'amélioration du kNN par rapport à la baseline.

# Baseline accuracy : 0.3333.

# kNN test accuracy : 0.9556.

# Différence : (0.9556 - 0.3333) * 100.

# Résultat : environ 62.22 points de pourcentage.

print(f"Baseline test accuracy : {baseline_acc:.2%}")

# Affiche l'accuracy de la baseline en pourcentage.

print(f"kNN test accuracy : {test_acc:.2%}")

# Affiche l'accuracy de kNN sur Test en pourcentage.

print(f"kNN train accuracy : {train_acc:.2%}")

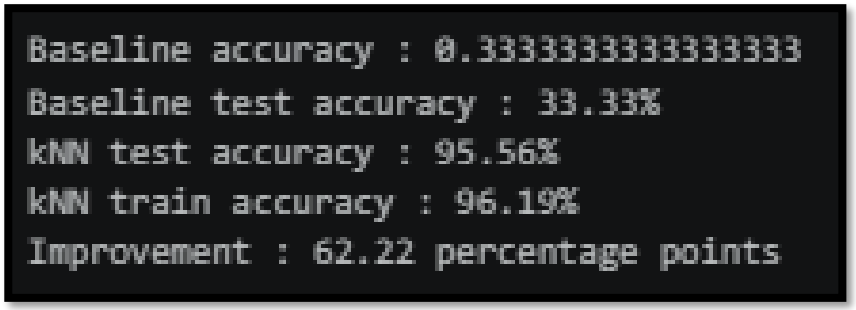
# Affiche l'accuracy de kNN sur Train en pourcentage.

print(f"Improvement : {improvement:.2f} percentage points")

# Affiche l'amélioration en points de pourcentage.

# :.2f : affiche 2 chiffres après la virgule.

# :.2% : convertit une proportion en pourcentage.
```



Modèle	kNN (k=3)	Accuracy Test
DummyClassifier (Baseline)	—	33,33 %
kNN (k=3)	96,19 %	95,56 %

Comprendre les résultats

Pourquoi kNN obtient-il une meilleure accuracy que la baseline ?

La baseline prédit systématiquement Setosa, sans tenir compte des caractéristiques de chaque fleur. En revanche, kNN utilise les valeurs des quatre caractéristiques pour rechercher des fleurs similaires et déterminer leur espèce.

Pourquoi calculer l'accuracy sur Train et sur Test ?

L'accuracy sur Train indique les performances du modèle sur les observations utilisées pendant l'entraînement. L'accuracy sur Test mesure ses performances sur des observations non utilisées pendant l'entraînement.

C'est donc 95,56 % (Test Accuracy) qui constitue le résultat principal à rapporter.

Pourquoi 62,22 points de pourcentage et non 62,22 % ?

Nous effectuons une soustraction entre deux pourcentages :

$$95,56\% - 33,33\% = 62,22 \text{ points}$$

Il s'agit d'une différence absolue en points de pourcentage, et non d'un taux d'augmentation relatif.

Task 5.3 — Which errors? Confusion matrix and misclassified samples

L'objectif: comprendre quelles erreurs le modèle a commises et quelles espèces il a confondues.

Question 1 : Confusion Matrix

Consigne du TP :

TODO: confusion_matrix(y_test, y_pred) shown as a DataFrame with species names (rows = true species, columns = predicted species).

Le code:

```
cm = confusion_matrix(y_test, y_pred)

# confusion_matrix() : construit une matrice de confusion.

# y_test : les vraies classes des 45 fleurs de test.

# y_pred : les classes prédites par kNN.

# Les lignes représentent les classes réelles (True).

# Les colonnes représentent les classes prédites (Predicted).

# Le résultat est une matrice NumPy de shape (3, 3).

cm_df = pd.DataFrame(

   cm,

   index=iris.target_names,

   columns=iris.target_names

)

# pd.DataFrame() : convertit la matrice NumPy en tableau Pandas.

# cm : contient les nombres de prédictions.

# index : définit les noms des lignes (classes réelles).
```

columns : définit les noms des colonnes (classes prédites).

iris.target_names : contient les noms des 3 espèces :

setosa, versicolor et virginica.

print(cm_df)

Affiche la matrice de confusion avec les noms

des espèces au lieu des codes numériques 0, 1, 2.

	setosa	versicolor	virginica
setosa	15	0	0
versicolor	0	15	0
virginica	0	2	13

⇨ Comment interpréter cette matrice ?

- Setosa : 15 fleurs correctement classées, aucune erreur.
- Versicolor : 15 fleurs correctement classées, aucune erreur.
- Virginica : 13 fleurs correctement classées et 2 fleurs incorrectement prédites comme Versicolor.

La diagonale principale contient les prédictions correctes : 15 + 15 + 13 = 43.

Les éléments hors diagonale représentent les erreurs : 2.

$$Accuracy = \frac{43}{45} \times 100 = 95,56\%$$

⇨ Cette observation confirme notre prédiction de la Task 4 (Q4) : le modèle risque de confondre Versicolor et Virginica, car leurs caractéristiques présentent des chevauchements.

Question 2 : List the misclassified test samples

TODO: list the misclassified test samples (features, true and predicted species).

Il faut identifier les fleurs mal classées, afficher leurs quatre caractéristiques et comparer l'espèce réelle à l'espèce prédite.

Le code :

errors = np.where(y_test != y_pred)[0]

y_test != y_pred : compare les classes réelles avec les classes prédites, élément par élément.

True : indique une prédiction incorrecte.

False : indique une prédiction correcte.

np.where() : trouve les positions des valeurs True.

[0] : récupère le tableau des indices des erreurs.

errors : contient les positions des fleurs mal classées.

errors_df = pd.DataFrame(

X_test[errors],

columns=features

)

X_test[errors] : sélectionne les observations mal classées.

pd.DataFrame() : crée un tableau Pandas.

columns=features : attribue les noms des 4 caractéristiques.

Chaque ligne correspond à une fleur mal classée.

errors_df["True species"] = iris.target_names[y_test[errors]]

y_test[errors] : sélectionne les vraies classes des erreurs.

iris.target_names : convertit les codes en noms d'espèces.

Crée une colonne contenant les espèces réelles.

errors_df["Predicted species"] = iris.target_names[y_pred[errors]]

y_pred[errors] : sélectionne les classes prédites.

Convertit ces codes en noms d'espèces.

Crée une colonne contenant les prédictions incorrectes.

print(errors_df)

Affiche les caractéristiques des fleurs mal classées

avec les vraies espèces et les espèces prédites.

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	6.3	2.8	5.1	1.5
1	4.9	2.5	4.5	1.7

	True species	Predicted species
0	virginica	versicolor
1	virginica	versicolor

⇨ Ces deux observations correspondent aux erreurs du modèle kNN avec le découpage demandé.

Question 3 — Visualiser les erreurs sur un graphique (optionnel)

Consigne du TP :

TODO (if time): redraw the petal scatter plot and mark the misclassified test samples with a large black 'x' (`marker="x", s=150, c="k"`).

Le code:

plt.figure(figsize=(8, 5))

Crée une figure de 8 pouces de largeur et 5 pouces de hauteur.

for name, group in df.groupby("species"):

Regroupe les observations selon l'espèce.

name : nom de l'espèce.

group : données correspondant à cette espèce.

plt.scatter(

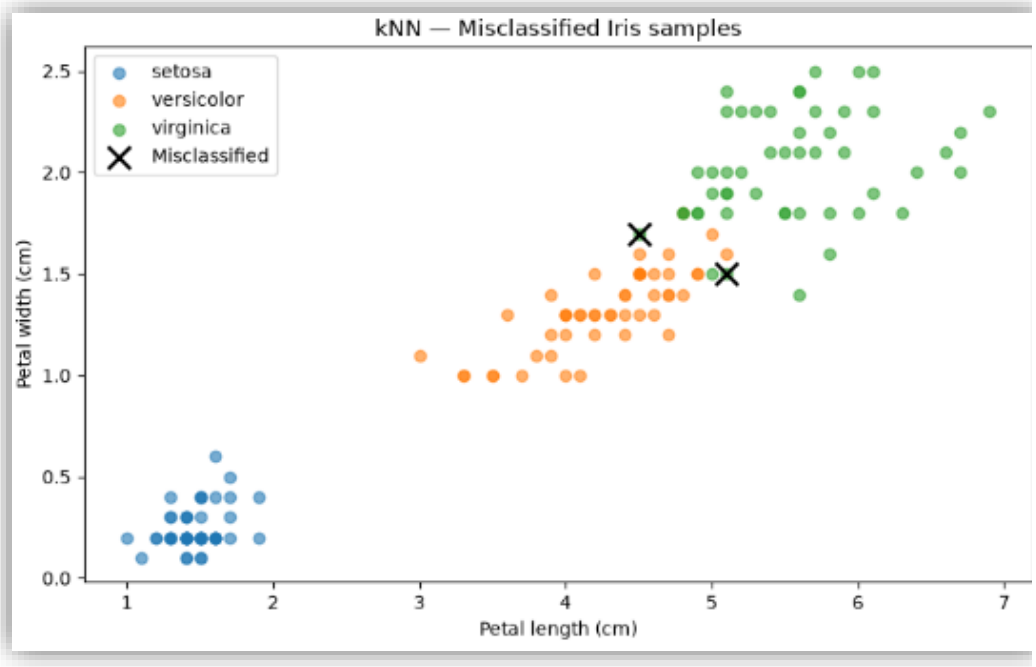
```
        group["petal length (cm)"],
        group["petal width (cm)"],
        label=name,
        alpha=0.6
    )
    # scatter() : crée un nuage de points.
    # Axe X : longueur des pétales.
    # Axe Y : largeur des pétales.
    # label=name : ajoute le nom à la légende.
    # alpha=0.6 : règle la transparence des points.
plt.scatter(
    X_test[errors, 2],
    X_test[errors, 3],
    marker="x",
    s=150,
    c="k",
    linewidths=2,
    label="Misclassified"
)
# X_test[errors, 2] : longueur des pétales mal classés.
# X_test[errors, 3] : largeur des pétales mal classés.
# marker="x" : utilise une croix comme marqueur.
# s=150 : définit la taille des croix.
# c="k" : sélectionne la couleur noire.
# linewidths=2 : définit l'épaisseur des croix.
# label : ajoute les erreurs à la légende.
plt.xlabel("Petal length (cm)")
# Définit l'étiquette de l'axe X.
plt.ylabel("Petal width (cm)")
# Définit l'étiquette de l'axe Y.

plt.title("kNN — Misclassified Iris samples")
# Définit le titre du graphique.
plt.legend()
# Affiche la légende des espèces et des erreurs.
plt.tight_layout()
```

Ajuste les espacements du graphique.

plt.show()

Affiche le graphique final.



⇨ Les deux croix noires correspondent aux deux fleurs **Virginica** classées incorrectement comme **Versicolor**. Elles se trouvent dans des régions où les caractéristiques des deux espèces peuvent être proches.

Task 5 — Q5 (a)

Question :

Why is the baseline exactly 1/3 here? => Pourquoi l'accuracy de la baseline est-elle exactement égale à 1/3 dans notre TP Iris ?

Dans notre ensemble d'entraînement (Train), nous avons :

- 35 Setosa
- 35 Versicolor
- 35 Virginica

Avec `DummyClassifier(strategy="most_frequent")`, le modèle choisit Setosa (classe 0), car les trois classes ont la même fréquence et Setosa possède la plus petite étiquette.

Dans l'ensemble de test, nous avons 45 fleurs : 15 Setosa, 15 Versicolor et 15 Virginica.

La baseline prédit toujours Setosa. Elle obtient donc 15 prédictions correctes sur 45.

$$Accuracy_{baseline} = \frac{15}{45} = \frac{1}{3} = 33,33\%$$

À retenir

⇨ L'accuracy de cette baseline correspond à la proportion, dans Test, de la classe choisie comme majoritaire dans Train.

Task 5 — Q5 (b)

Question :

Does kNN beat the baseline, and by how much? => Le modèle kNN est-il plus performant que la baseline ? Si oui, de combien ?

⇨ Nous avons obtenu les résultats suivants :

Modèle	Test Accuracy
Baseline (`DummyClassifier`)	33,33 %
kNN (`n_neighbors=3`)	95,56 %

Le modèle kNN est plus performant que la baseline, car il réalise davantage de prédictions correctes.

Calcul de l'amélioration

$$Improvement = Accuracy_{kNN} - Accuracy_{baseline}$$

$$= 95,56\% - 33,33\%$$

$$\boxed{62,22 \text{ points de pourcentage}}$$

Le code :

```
improvement = (test_acc - baseline_acc) * 100
# test_acc : accuracy de kNN sur Test (0.9556).
# baseline_acc : accuracy de la baseline sur Test (0.3333).
# La soustraction mesure la différence de performance.
# * 100 : convertit cette différence en points de pourcentage.

print(f"Improvement: {improvement:.2f} percentage points")
# Affiche : Improvement: 62.22 percentage points.
```

```
Improvement: 62.22 percentage points
```

Task 5 — Q5 (c)

Question :

Do the errors match your prediction in Q4? => Les erreurs de classification observées correspondent-elles à ta prédiction formulée dans la question Q4 ?

1. Explication

Dans la Task 4 (Q4), nous avons analysé les graphiques des caractéristiques des fleurs Iris.

Nous avions prédit que :

- Setosa serait facilement reconnue, car ses caractéristiques sont bien distinctes.
- Versicolor et Virginica seraient plus difficiles à distinguer, en raison du chevauchement de leurs caractéristiques.

Dans la Task 5, la matrice de confusion du modèle kNN a montré :

Espèce réelle	Espèce prédite	Nombre
Setosa	Setosa	15
Versicolor	Versicolor	15
Virginica	Virginica	13
Virginica	Versicolor	2 erreurs

⇨ les erreurs correspondent à notre prédiction de Q4. Les deux erreurs concernent des fleurs Virginica classées comme Versicolor. Aucune fleur Setosa n'a été mal classée

À retenir

Le chevauchement des caractéristiques (feature overlap) peut rendre certaines classes difficiles à distinguer et provoquer des erreurs de

Task 5 — Q5 (d)

Question :

Why did we stratify the split? => Pourquoi avons-nous utilisé la stratification lors de la séparation des données en Train et Test ?

Explication

La stratification (Stratified Split) est une technique qui permet de conserver approximativement les mêmes proportions de classes dans les ensembles d'entraînement (Train) et de test (Test).

Dans notre dataset Iris, nous avons 150 fleurs réparties équitablement entre trois espèces.

Espèce	Dataset initial	Train (70 %)	Test (30 %)
Setosa	50	35	15
Versicolor	50	35	15
Virginica	50	35	15
Total	150	105	45

Chaque espèce représente 33,33 % des observations dans les trois ensembles.

Le code :

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=SEED, stratify=y
)
# train_test_split() : sépare les données en Train et Test.
# test_size=0.3 : réserve 30 % des observations pour Test.
# random_state=SEED : rend le découpage reproductible.
# stratify=y : conserve approximativement les proportions des classes de y dans Train et Test.
# Cela permet d'obtenir une répartition représentative des différentes espèces dans les deux ensembles.
```

Sans stratify=y : le découpage aléatoire peut produire des proportions différentes entre Train et Test.

Avec stratify=y : les proportions des classes sont préservées autant que possible, ce qui facilite une évaluation représentative du modèle.

Stratified Split : méthode de séparation des données qui conserve approximativement la distribution des classes dans les ensembles Train et Test.

La stratification est particulièrement importante lorsque le dataset est déséquilibré (imbalanced dataset).

Task 5 — Q5 (e)

Question :

Why is the train accuracy not the number to report? => Pourquoi l'accuracy calculée sur l'ensemble d'entraînement (Train) n'est-elle pas celle qu'il faut rapporter comme résultat principal ?

Explication

En Machine Learning, nous distinguons deux types d'accuracy :

- Train Accuracy : mesure les performances du modèle sur les données utilisées pendant l'entraînement.
- Test Accuracy : mesure les performances du modèle sur de nouvelles données, jamais utilisées pendant l'entraînement.

L'objectif du Machine Learning est de construire un modèle capable de généraliser (Generalization), c'est-à-dire de réaliser de bonnes prédictions sur des données non vues.

Métrique	Résultat kNN	Signification
Train Accuracy	96,19 %	Performance sur les données d'entraînement
Test Accuracy	95,56 %	Performance sur les données non vues

La Train Accuracy peut être trop optimiste, car le modèle a déjà utilisé ces observations pendant son entraînement.

La Test Accuracy permet de mieux estimer sa capacité de généralisation.

Task 6 — Transfer: Package and Reuse the Protocol

Objectif : transformer le protocole d'évaluation de la Task 5 en une fonction Python réutilisable, puis appliquer cette même fonction à deux datasets : Iris et Wine.

Dans cette tâche, nous allons apprendre à utiliser `def`, `return`, les paramètres d'une fonction et les dictionnaires Python, tout en conservant une évaluation correcte du Machine Learning.

Avant de commancer :

Wine Dataset — Définition

Wine signifie « vin » en anglais.

Le Wine Dataset est un jeu de données (dataset) disponible dans la bibliothèque **scikit-learn**, utilisé pour apprendre et tester des algorithmes de Machine Learning, notamment la classification.

Il contient des résultats d'analyses chimiques de vins issus de trois cépages (cultivars) cultivés dans une même région d'Italie.

1. Caractéristiques du « Wine Dataset »

Élément	Description
Nombre d'observations	178 vins
Nombre de features	13 caractéristiques chimiques
Nombre de classes	3
Target	Type de cépage (0, 1 ou 2)
Type de problème	Classification supervisée

2. Exemples de caractéristiques (Features)

- Alcohol : teneur en alcool.
- Magnesium : quantité de magnésium.
- Hue : mesure liée à la teinte du vin.
- Proline : teneur en proline, un acide aminé.
- Color intensity : intensité de la couleur.

3. Quel est l'objectif en Machine Learning ?

À partir des 13 caractéristiques chimiques d'un vin, le modèle doit prédire à laquelle des trois classes de cépages il appartient.

4. Différence entre Iris et Wine

Iris	Wine
Classification de fleurs	Classification de vins

150 observations	178 observations
4 caractéristiques	13 caractéristiques
3 espèces	3 classes de cépages
Classes équilibrées	Classes déséquilibrées

À retenir : Wine est un dataset de classification supervisée permettant de prédire le type de cépage d'un vin à partir de ses caractéristiques chimiques.

Dans notre Task 6, nous l'utilisons pour vérifier si le modèle kNN, performant sur Iris, reste efficace sur un autre dataset.

Question 6.1 — Comprendre et définir la fonction `honest_evaluation()`

Le Code fourni dans le TP :

```
def honest_evaluation(X, y, k=3, test_size=0.3, seed=SEED):
    """Stratified split, majority baseline and kNN;
    return the two TEST accuracies in a dict."""
    # TODO
```

1. Quel est l'objectif ?

Créer **une fonction Python** qui réalise automatiquement les opérations suivantes :

1. Séparer les données en Train et Test avec stratification.
2. Entraîner un `DummyClassifier` et calculer son Test Accuracy.
3. Entraîner un modèle kNN et calculer son Test Accuracy.
4. Retourner les deux accuracies dans un dictionnaire Python.

L'avantage est de pouvoir utiliser la même fonction sur Iris et Wine sans réécrire le protocole d'évaluation.

2. Comprendre les paramètres

Paramètre	Définition
`X`	Matrice des caractéristiques (features)
`y`	Vecteur des classes réelles (target)
`k=3`	Nombre de voisins du modèle kNN
`test_size=0.3`	30 % des observations réservées au test
`seed=SEED`	Nombre utilisé pour rendre le découpage reproductible

3. Première étape : compléter le Split

Pour commencer, nous allons uniquement compléter la partie qui sépare les données en Train et Test

Le code :

```
# ============================================================
# TASK 6 — HONEST EVALUATION
# ============================================================

def honest_evaluation(X, y, k=3, test_size=0.3, seed=SEED):
    # def : permet de définir une fonction Python.
    # X : matrice des caractéristiques (features).
```

```
    # y : vecteur des classes réelles (target).
    # k=3 : nombre de voisins utilisés par kNN.
    # test_size=0.3 : réserve 30 % des données pour Test.
    # seed=SEED : permet de reproduire le même découpage.

    """Stratified split, majority baseline and kNN;
    return the two TEST accuracies in a dict."""
    # Docstring : décrit le rôle de la fonction.
    # dict : dictionnaire Python contenant les résultats.

    # ============================================================
    # ÉTAPE 1 — STRATIFIED SPLIT
    # ============================================================

    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=test_size,
        random_state=seed,
        stratify=y
    )
    # train_test_split() : sépare les données en Train et Test.
    # test_size=test_size : définit la proportion du test.
    # random_state=seed : rend le découpage reproductible.
    # stratify=y : conserve les proportions des classes.
    # X_train, y_train : données d'entraînement.
    # X_test, y_test : données de test.

    # ============================================================
    # ÉTAPE 2 — BASELINE : DUMMYCLASSIFIER
    # ============================================================

    baseline = DummyClassifier(strategy="most_frequent")
    # Crée un modèle de référence.
    # most_frequent : prédit toujours la classe majoritaire
```

```
    # déterminée sur l'ensemble d'entraînement.

    baseline.fit(X_train, y_train)
    # Ajuste la baseline sur Train.
    # Identifie la classe la plus fréquente dans y_train.

    baseline_acc = baseline.score(X_test, y_test)
    # Calcule l'accuracy de la baseline sur Test.
    # Compare les prédictions aux vraies classes.

    #
==========================================================
    # ÉTAPE 3 — MODÈLE KNN
    #
==========================================================

    knn = KNeighborsClassifier(n_neighbors=k)
    # Crée un classificateur kNN.
    # n_neighbors=k : utilise k plus proches voisins.
    # Par défaut, k=3 et la distance est euclidienne.

    knn.fit(X_train, y_train)
    # Ajuste kNN sur les données Train.
    # Conserve les observations pour rechercher les voisins.

    knn_acc = knn.score(X_test, y_test)
    # Calcule l'accuracy de kNN sur Test.
    # Le modèle est évalué sur des données non vues.

    #
==========================================================
    # ÉTAPE 4 — RETURN RESULTS
    #
==========================================================

    return {
        "baseline_accuracy": baseline_acc,
        "knn_accuracy": knn_acc
```

```
    }
    # return : renvoie les résultats de la fonction.
    # {} : crée un dictionnaire contenant deux éléments.
    # baseline_accuracy : accuracy de DummyClassifier.
    # knn_accuracy : accuracy du modèle kNN.
    # Les deux valeurs sont calculées sur le même Test.



# ============================================================
# ÉTAPE 5 — APPLICATION AU DATASET IRIS
# ============================================================

iris_results = honest_evaluation(X, y)
# Appelle la fonction avec les données Iris.
# X : caractéristiques des 150 fleurs.
# y : classes réelles des fleurs.
# Utilise les paramètres par défaut de la fonction.

print("Iris results :", iris_results)
# Affiche les deux accuracies de Iris.
# Baseline : 33.33%.
# kNN : 95.56%.
# ============================================================
# TASK 6 — HONEST EVALUATION
# ============================================================
def honest_evaluation(X, y, k=3, test_size=0.3, seed=SEED):
    # def : permet de définir une fonction Python.
    # X : matrice des caractéristiques (features).
    # y : vecteur des classes réelles (target).
    # k=3 : nombre de voisins utilisés par kNN.
    # test_size=0.3 : réserve 30 % des données pour Test.
    # seed=SEED : permet de reproduire le même découpage.
    """Stratified split, majority baseline and kNN;
    return the two TEST accuracies in a dict."""
```

```
# ÉTAPE 1 — STRATIFIED SPLIT
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=test_size,
    random_state=seed,
    stratify=y
)
# train_test_split() : sépare les données en Train et Test.
# test_size=test_size : définit la proportion du test.
# random_state=seed : rend le découpage reproductible.
# stratify=y : conserve les proportions des classes.
# X_train, y_train : données d'entraînement.
# X_test, y_test : données de test.
# ÉTAPE 2 — BASELINE : DUMMYCLASSIFIER
baseline = DummyClassifier(strategy="most_frequent")
# Crée un modèle de référence.
# most_frequent : prédit toujours la classe majoritaire
# déterminée sur l'ensemble d'entraînement.
baseline.fit(X_train, y_train)
# Ajuste la baseline sur Train.
# Identifie la classe la plus fréquente dans y_train.
baseline_acc = baseline.score(X_test, y_test)
# Calcule l'accuracy de la baseline sur Test.
# Compare les prédictions aux vraies classes.
# ÉTAPE 3 — MODÈLE KNN
knn = KNeighborsClassifier(n_neighbors=k)
# Crée un classificateur kNN.
# n_neighbors=k : utilise k plus proches voisins.
# Par défaut, k=3 et la distance est euclidienne.
knn.fit(X_train, y_train)
# Ajuste kNN sur les données Train.
# Conserve les observations pour rechercher les voisins.
knn_acc = knn.score(X_test, y_test)
# Calcule l'accuracy de kNN sur Test.
# Le modèle est évalué sur des données non vues.
```

```
    # ÉTAPE 4 — RETURN RESULTS
    return {
        "baseline_accuracy": baseline_acc,
        "knn_accuracy": knn_acc
    }
    # return : renvoie les résultats de la fonction.
    # {} : crée un dictionnaire contenant deux éléments.
    # baseline_accuracy : accuracy de DummyClassifier.
    # knn_accuracy : accuracy du modèle kNN.
    # Les deux valeurs sont calculées sur le même Test.
# ÉTAPE 5 — APPLICATION AU DATASET IRIS
iris_results = honest_evaluation(X, y)
# Appelle la fonction avec les données Iris.
# X : caractéristiques des 150 fleurs.
# y : classes réelles des fleurs.
# Utilise les paramètres par défaut de la fonction.
print("Iris results :", iris_results)
# Affiche les deux accuracies de Iris.
# Baseline : 33.33%.
# kNN : 95.56%.
# ÉTAPE 6 — APPLICATION AU DATASET WINE
wine = load_wine(as_frame=True)
# Charge le dataset Wine depuis scikit-learn.
# Il contient 178 vins, 13 caractéristiques et 3 classes.
X_wine = wine.data.to_numpy()
# Extrait les 13 caractéristiques chimiques des vins.
# Convertit les données en tableau NumPy.
# Shape : (178, 13).
y_wine = wine.target.to_numpy()
# Extrait les classes réelles des 178 vins.
# Shape : (178,).
wine_results = honest_evaluation(X_wine, y_wine)
# Applique exactement le même protocole au dataset Wine.
# Effectue le split, la baseline et l'évaluation kNN.
print("Wine results :", wine_results)
```

```
# Affiche les deux accuracies sur Wine.

# Baseline : 38.89%.

# kNN : 68.52%.

# ÉTAPE 7 — WINE CLASS COUNTS

print("Wine class counts :", np.bincount(y_wine))

# np.bincount() : compte les éléments de chaque classe.

# Résultat : [59 71 48].

# Classe 0 : 59 vins.

# Classe 1 : 71 vins.

# Classe 2 : 48 vins.

# ÉTAPE 8 — MIN / MAX DES CARACTÉRISTIQUES WINE

for feature in ["hue", "magnesium", "proline"]:

    # Parcourt les trois caractéristiques chimiques.

    # feature : contient le nom de la colonne courante.

    print(feature,

        "min =", wine.frame[feature].min(),

        "max =", wine.frame[feature].max())

    # wine.frame[feature] : sélectionne la colonne.

    # min() : retourne la valeur minimale.

    # max() : retourne la valeur maximale.

    # print() : affiche les valeurs obtenues.
```

```
Iris results : {'baseline_accuracy': 0.3333333333333333, 'knn_accuracy': 0.9555555555555556}
Wine results : {'baseline_accuracy': 0.3888888888888889, 'knn_accuracy': 0.6851851851851852}
Wine class counts : [59 71 48]
hue min = 0.48 max = 1.71
magnesium min = 70.0 max = 162.0
proline min = 278.0 max = 1680.0
```

Structure du programme

Étape	Fonction
1	Séparer les données Train/Test
2	Créer et évaluer la baseline
3	Entraîner et évaluer kNN
4	Retourner les accuracies dans un dictionnaire
5	Tester la fonction sur Iris
6	Réutiliser la fonction sur Wine
7	Compter les classes de Wine
8	Afficher les valeurs Min/Max

⇨ La fonction honest_evaluation() reste identique pour les deux datasets : seuls les arguments X et y changent. C'est le principe de réutilisabilité du code (Code Reusability).

⇨ La fonction exécute toujours les quatre étapes :

1. Split : séparer les données en Train et Test.
2. Baseline : entraîner et évaluer DummyClassifier.
3. kNN : entraîner et évaluer KNeighborsClassifier.

4. Return : retourner les deux accuracies.

Task 7 — Code Review: Too Good to Be True

Objectif : apprendre à détecter les erreurs dans un protocole d'évaluation de Machine Learning et comprendre pourquoi une accuracy très élevée n'est pas toujours fiable.

Dans cette tâche, nous allons analyser deux erreurs présentes dans `buggy_pipeline.py`, puis corriger le protocole.

Que signifie buggy_pipeline ?

- **Buggy** : programme contenant des erreurs.
- **Pipeline** : enchaînement d'étapes de traitement, d'entraînement et d'évaluation.

⇨ Exécuter le fichier `buggy_pipeline.py`

Consigne du TP :

Run the script from the notebook and read it line by line.

!python buggy_pipeline.py

! : permet d'exécuter une commande système depuis Jupyter.

python : lance l'interpréteur Python.

buggy_pipeline.py : fichier Python contenant

deux erreurs dans le protocole d'évaluation.

```
!python buggy_pipeline.py
✓ 12.2s

Report A - kNN (k=1) accuracy: 1.000
Report B - baseline accuracy: 0.000
Report B - kNN (k=3) accuracy: 0.622
Report B - improvement over baseline: 62.2 points
```

Task 7 — Étape 2 : Analyse des résultats :

Nous allons commencer par Report A uniquement :

Cela signifie que le modèle a correctement classé 100 % des observations sur lesquelles il a été évalué.

Nous devons vérifier si le modèle a été évalué sur les mêmes données que celles utilisées pendant son entraînement.

Mais attention : ce résultat ne prouve pas que le modèle sera aussi performant sur de nouvelles données.

Étape suivante — Examiner le code de Report A

Ouvre le fichier `buggy_pipeline.py` dans VS Code et cherche ces instructions :

```python
# --------------- Report A ---------------
model = KNeighborsClassifier(n_neighbors=1)
model.fit(X, y)
acc_a = model.score(X, y)
print(f"Report A - kNN (k=1) accuracy: {acc_a:.3f}")
```

```
# ---------- Report A ----------
model = KNeighborsClassifier(n_neighbors=1)
# Crée un classificateur kNN utilisant un seul voisin.
# k=1 : la classe du voisin le plus proche est prédite.
model.fit(X, y)
# Entraîne le modèle sur toutes les 150 fleurs Iris.
# X : les caractéristiques des fleurs.
# y : les classes réelles des fleurs.
acc_a = model.score(X, y)
# Calcule l'accuracy sur les mêmes données X et y
# qui ont été utilisées pendant l'entraînement.
# ERREUR : aucune donnée indépendante n'est utilisée
# pour évaluer la capacité de généralisation du modèle.
print(f"Report A - kNN (k=1) accuracy: {acc_a:.3f}")
# Affiche l'accuracy avec 3 chiffres après la virgule.
# Résultat : 1.000, soit 100 %.
```

L'erreur est dans :

```
acc_a = model.score(X, y)
```

Pourquoi ?

- `model.fit(X, y)` : le modèle utilise les 150 fleurs pour s'entraîner.
- `model.score(X, y)` : il est ensuite évalué sur ces mêmes 150 fleurs.

Avec `k=1`, chaque observation peut être son propre voisin le plus proche, ce qui explique le score parfait sur ce dataset.

Le problème : le modèle n'a jamais été évalué sur de nouvelles observations.

à retenir

Training-set evaluation (évaluation sur Train) : évaluation d'un modèle sur les mêmes données que celles utilisées pendant son entraînement. Elle peut produire une accuracy trop optimiste et ne permet pas de mesurer correctement la généralisation.

On passe au Report B :

```
# ---------- Report B ----------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, shuffle=False
)

baseline = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
knn = KNeighborsClassifier(n_neighbors=3).fit(X_train, y_train)

acc_base = baseline.score(X_test, y_test)
acc_knn = knn.score(X_test, y_test)
print(f"Report B - baseline accuracy: {acc_base:.3f}")
print(f"Report B - kNN (k=3) accuracy: {acc_knn:.3f}")
print(f"Report B - improvement over baseline: {100 * (acc_knn - acc_base):.1f} points")

# Hint for the reviewer (uncomment when you are ready):
# print("train class counts:", np.bincount(y_train, minlength=3))
# print("test  class counts:", np.bincount(y_test, minlength=3))
```

Le code expliquer :

```
# ---------- Report B ----------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, shuffle=False
)
# train_test_split() : sépare les données en Train et Test.
# test_size=0.3 : réserve 30 % des données pour Test.
# shuffle=False : conserve l'ordre initial des observations.
# ERREUR : Iris est organisé par espèce.
# Les observations ne sont donc pas mélangées.
# Test contient uniquement des fleurs Virginica.
baseline = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
# Crée une baseline qui prédit toujours la classe
# majoritaire dans Train.
# fit() : détermine cette classe.
knn = KNeighborsClassifier(n_neighbors=3).fit(X_train, y_train)
# Crée un classificateur kNN utilisant 3 voisins.
# fit() : ajuste le modèle sur Train.
acc_base = baseline.score(X_test, y_test)
# Calcule l'accuracy de la baseline sur Test.
# Résultat : 0.000, soit 0 %.
acc_knn = knn.score(X_test, y_test)
# Calcule l'accuracy de kNN sur Test.
# Résultat : environ 0.622, soit 62.22 %.
print(f"Report B - baseline accuracy: {acc_base:.3f}")
# Affiche l'accuracy de la baseline.
print(f"Report B - kNN (k=3) accuracy: {acc_knn:.3f}")
# Affiche l'accuracy de kNN.
print(f"Report B - improvement over baseline: {100 * (acc_knn - acc_base):.1f} points")
# Calcule la différence entre les deux accuracies.
# Résultat : 62.2 points de pourcentage.
```

2. Où se trouve l'erreur ?

L'instruction responsable est :

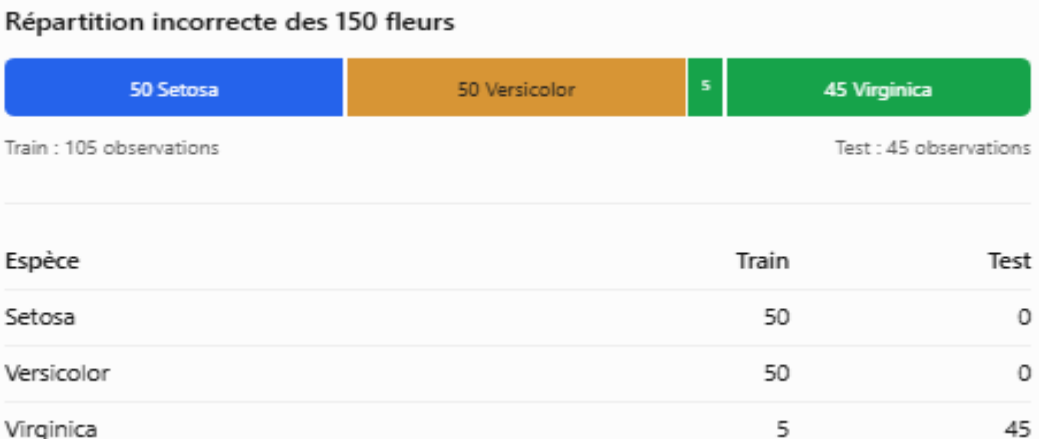
```
X, y, test_size=0.3, shuffle=False
```

`shuffle=False` signifie que les observations ne sont pas mélangées avant le découpage.

Or, le dataset Iris est initialement ordonné :

Positions	Espèce
0 à 49	Setosa
50 à 99	Versicolor
100 à 149	Virginica

Avec 70 % pour Train et 30 % pour Test :



Espèce	Train	Test
Setosa	50	0
Versicolor	50	0
Virginica	5	45

⇨ Le problème est que Test contient uniquement Virginica, alors que Train contient principalement Setosa et Versicolor.

La baseline prédit toujours Setosa (classe 0), qui est l'une des classes majoritaires de Train.

Comme Test ne contient aucune Setosa :

$$Accuracy_{baseline} = \frac{0}{45} = 0\%$$

Le modèle kNN obtient 62,22 %, mais son évaluation concerne uniquement les Virginica. Ce score ne représente donc pas correctement sa performance sur l'ensemble des trois espèces.

4. Comment vérifier cette erreur dans le script ?

À la fin de `buggy_pipeline.py`, tu as deux lignes commentées Retire simplement le `#` :

```
print("train class counts:", np.bincount(y_train, minlength=3))

# Affiche le nombre d'observations de chaque classe dans Train.

# Résultat attendu : [50 50 5].

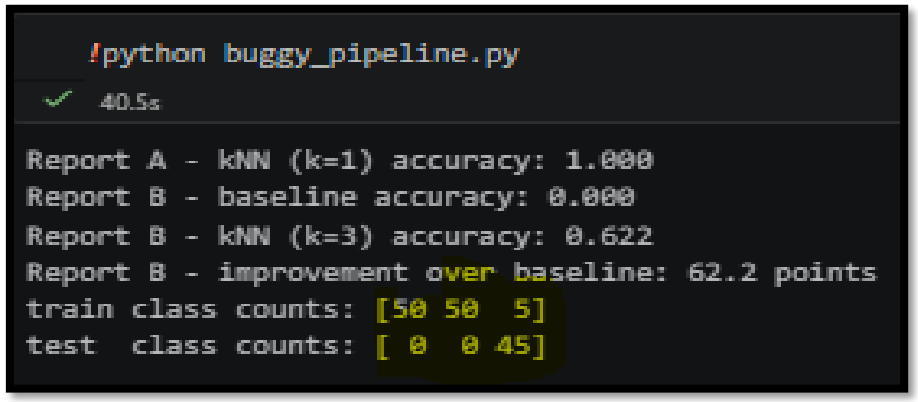
print("test class counts:", np.bincount(y_test, minlength=3))

# Affiche le nombre d'observations de chaque classe dans Test.

# Résultat attendu : [0 0 45].
```

À retenir

une séparation Train/Test doit produire des ensembles adaptés à l'objectif d'évaluation. Pour notre TP Iris, nous utilisons `shuffle=True`, `stratify=y` et `random_state=42`.



- ⇨ **Le problème** : toutes les fleurs du Test sont des Virginica.à cause de shuffle=False
- ⇨ La baseline obtient 0 %, tandis que kNN obtient 62,22 %. Ces scores ne permettent pas d'évaluer correctement les performances sur les trois espèces.

Étape 6 : Corriger le découpage Train/Test

Consigne : Write the corrected evaluation here (shuffled + stratified split with random_state=SEED).

Au dessous de la cellule : !python buggy_pipline.py

Metter : # CORRECT TRAIN / TEST SPLIT

```
X = df[features].to_numpy()

# Récupère les 4 caractéristiques du dataset Iris.

y = df["target"].to_numpy()

# Récupère les classes réelles des fleurs.

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=SEED,
    shuffle=True,
    stratify=y
)

# test_size=0.3 : réserve 30 % des fleurs pour Test.

# random_state=SEED : rend le découpage reproductible.

# shuffle=True : mélange les observations avant le split.

# stratify=y : conserve les proportions des trois espèces.

print("Train class counts:", np.bincount(y_train))

# Affiche le nombre de fleurs par espèce dans Train.

print("Test class counts:", np.bincount(y_test))

# Affiche le nombre de fleurs par espèce dans Test.
```



ÉTAPE 2 — CORRECT BASELINE EVALUATION

```
baseline = DummyClassifier(strategy="most_frequent")
```

Crée un classificateur de référence.

Il prédit toujours la classe majoritaire de Train.

```
baseline.fit(X_train, y_train)
```

Ajuste la baseline uniquement sur Train.

Les 3 espèces possèdent chacune 35 observations.

Le modèle choisit donc Setosa (classe 0).

```
baseline_acc = baseline.score(X_test, y_test)
```

Calcule l'accuracy sur Test uniquement.

La baseline prédit correctement 15 fleurs sur 45.

```
print(f"Corrected Baseline Accuracy: {baseline_acc:.2%}")
```

Affiche l'accuracy de la baseline en pourcentage.



Consigne : entraîner le modèle `KNeighborsClassifier(n_neighbors=3)` uniquement sur Train, puis calculer son accuracy sur Test.

ÉTAPE 3 — CORRECT KNN EVALUATION

```
knn = KNeighborsClassifier(n_neighbors=3)
```

Crée un classificateur kNN.

n_neighbors=3 : utilise les 3 plus proches voisins.

Le modèle prédit la classe majoritaire parmi ces voisins.

```
knn.fit(X_train, y_train)
```

Entraîne le modèle uniquement sur Train.

X_train : caractéristiques des 105 fleurs.

y_train : classes réelles de ces 105 fleurs.

```
knn_acc = knn.score(X_test, y_test)
```

Évalue le modèle uniquement sur Test.

X_test : caractéristiques des 45 fleurs non vues.

y_test : classes réelles des 45 fleurs.

```
# 43 prédictions correctes sur 45.

# Accuracy attendue : 0.9556, soit 95.56%.

print(f"Corrected kNN Accuracy: {knn_acc:.2%}")

# Affiche l'accuracy de kNN en pourcentage.

# :.2% : affiche deux chiffres après la virgule.
```

```
Train class counts: [35 35 35]
Test class counts: [15 15 15]
Corrected Baseline Accuracy: 33.33%
Corrected kNN Accuracy: 95.56%
```

Comparaison avec le Report B

Évaluation	kNN Test Accuracy
Report B incorrect (`shuffle=False`)	62,22 %
Évaluation corrigée	95,56 %

- ⇨ Dans notre évaluation corrigée, Test contient 15 Setosa, 15 Versicolor et 15 Virginica. Il permet donc d'évaluer le modèle sur les trois espèces.

À retenir

l'accuracy doit être calculée sur des données non utilisées pendant l'entraînement, avec un découpage représentatif du problème de classification.

```
# ÉTAPE 4 — IMPROVEMENT OVER BASELINE

improvement = (knn_acc - baseline_acc) * 100

# knn_acc : accuracy du modèle kNN sur Test (0.9556).

# baseline_acc : accuracy de la baseline sur Test (0.3333).

# Calcule la différence entre les deux accuracies.

# * 100 : exprime la différence en points de pourcentage.

# Résultat attendu : 62.22 points.

print(f"Corrected Improvement: {improvement:.2f} points")

# Affiche l'amélioration avec 2 chiffres après la virgule.

# Résultat attendu : Corrected Improvement: 62.22 points.
```

```
Train class counts: [35 35 35]
Test class counts: [15 15 15]
Corrected Baseline Accuracy: 33.33%
Corrected kNN Accuracy: 95.56%
Corrected Improvement: 62.22 points
```

Comparaison finale

Indicateur	Report B incorrect	Évaluation corrigée
Baseline Accuracy	0 %	33,33 %
kNN Accuracy	62,22 %	95,56 %
Improvement	+62,22 points	+62,22 points

⇨ **Remarque importante :** même si l'amélioration est identique, les performances sont différentes. Le Report B reste invalide, car son ensemble de test contient uniquement des Virginica.

8- Synthesis and Exit Ticket — Questions et réponses

Question : In at most four sentences, write the evaluation checklist you will apply to every model in this course.

⇨ Quelles sont les quatre règles à respecter pour évaluer correctement un modèle de Machine Learning ?

Réponse :

1. Data Splitting : séparer les données en Train et Test avec un découpage reproductible et stratifié lorsque c'est approprié.
2. Training : entraîner le modèle uniquement sur Train pour éviter le Data Leakage (fuite de données).
3. Evaluation : mesurer l'accuracy sur Test et la comparer à une baseline (modèle de référence).
4. Error Analysis : analyser la matrice de confusion pour identifier les classes mal prédites.

Exit Ticket

Q1 — Why is a test accuracy meaningless without a baseline?

Traduction : Pourquoi faut-il comparer l'accuracy du modèle à une baseline ?

Réponse : Parce que l'accuracy seule ne permet pas de savoir si le modèle est meilleur qu'une méthode de référence simple.

Q2 — A colleague announces 99% accuracy. Which two questions do you ask first?

Traduction : Un collègue annonce une accuracy de 99 %. Quelles sont les deux questions à lui poser ?

Réponse :

1. **Fiabilité de l'évaluation :** Le modèle a-t-il été évalué sur un ensemble de test indépendant, jamais utilisé pendant l'entraînement ou la sélection du modèle ?
2. **Comparaison avec la baseline :** L'accuracy obtenue est-elle supérieure à celle du modèle de référence évalué sur le même ensemble de test ?

Objectif : vérifier que les 99 % représentent une performance fiable et meilleure qu'une méthode simple.

Q3 — One thing from today that is still unclear to you.

Traduction : Quelle notion du TP reste encore difficile à comprendre ?

Exemple de réponse : Comment choisir la meilleure valeur de `k` avec la Cross-Validation sans utiliser les données Test ?

Explication : nous choisissons `k` grâce à la validation croisée sur Train, puis nous évaluons le modèle sélectionné une seule fois sur Test.

Optional Challenges — Exercices facultatifs

C1 — Majority-Class Baseline with NumPy

Question du TP :

‘’’Re-implement the majority-class baseline with NumPy only (`np.unique(..., return_counts=True)`) and check it matches `DummyClassifier`.’’’

Traduction : recréer une baseline qui prédit toujours la classe majoritaire, en utilisant uniquement NumPy, puis vérifier que son accuracy est identique à celle de `DummyClassifier`

Dans la Task 5, nous avons utilisé :

```
baseline = DummyClassifier(strategy="most_frequent")
```

⇨ Cette instruction permet de créer automatiquement un modèle qui prédit toujours la classe majoritaire de Train.

Dans C1, nous allons reproduire ce fonctionnement nous-mêmes avec NumPy, sans utiliser `DummyClassifier` pour effectuer les prédictions.

1 — MAJORITY-CLASS BASELINE WITH NUMPY

```
X_iris = df[features].to_numpy()
```

Récupère les classes réelles des 150 fleurs Iris.

2 — STRATIFIED TRAIN / TEST SPLIT

```
X_train, X_test, y_train, y_test = train_test_split(
X_iris, y_iris,
test_size=0.3,
random_state=SEED,
stratify=y_iris
)
```

Train : 105 observations.

Test : 45 observations.

3 — IDENTIFIER LA CLASSE MAJORITAIRE

```
classes, counts = np.unique(y_train, return_counts=True)
```

np.unique() : identifie les valeurs distinctes de y_train.

y_train : contient les classes des 105 fleurs Iris.

return_counts=True : compte le nombre d'observations appartenant à chaque classe.

classes : tableau des classes distinctes.

counts : tableau des effectifs correspondants.

```
print("Classes :", classes)
```

Affiche les trois classes : [0 1 2].

```
print("Counts :", counts)
```

Affiche le nombre d'observations par classe.

```
Classes : [0 1 2]
Counts : [35 35 35]
```

Lecture plus simple :

Classe	Espèce	Effectif
0	Setosa	35
1	Versicolor	35
2	Virginica	35

```
majority_class = classes[np.argmax(counts)]
```

np.argmax(counts) : trouve l'indice du plus grand effectif.

```
# En cas d'égalité, retourne le premier indice.

# Ici, les effectifs sont égaux : [35 35 35].

# La classe 0 (Setosa) est donc sélectionnée.


print("Majority class :", majority_class)

# Affiche : Majority class : 0.

4 — PRÉDIRE AVEC NUMPY

y_pred_numpy = np.full(y_test.shape, majority_class)

# np.full() : crée un tableau rempli d'une valeur donnée.

# y_test.shape : définit la forme du tableau (45,).

# majority_class : valeur utilisée pour les prédictions. Ici, les 45 prédictions
sont égales à 0 (Setosa).

print("Predictions :", y_pred_numpy)

# Affiche les 45 classes prédites.

print("Predictions shape :", y_pred_numpy.shape)

# Affiche la forme des prédictions : (45,).

5 — CALCULER L'ACCURACY AVEC NUMPY

correct_predictions = (y_pred_numpy == y_test)

# Compare chaque prédiction avec sa vraie classe.

# True : prédiction correcte.

# False : prédiction incorrecte.

# Produit un tableau booléen de 45 éléments.

numpy_acc = np.mean(correct_predictions)

# np.mean() : calcule la moyenne des valeurs booléennes.

# True est compté comme 1 et False comme 0.

# 15 prédictions correctes parmi 45.

# Résultat : 15 / 45 = 0.3333.

print(f"NumPy Baseline Accuracy : {numpy_acc:.2%}")

# Affiche : NumPy Baseline Accuracy : 33.33%.


6 — COMPARER AVEC DUMMYCLASSIFIER

baseline = DummyClassifier(strategy="most_frequent")

# Crée un classificateur de référence scikit-learn.

# Prédit toujours la classe majoritaire de Train.

baseline.fit(X_train, y_train)

# Identifie la classe majoritaire dans y_train.

y_pred_dummy = baseline.predict(X_test)

# Prédit les classes des 45 fleurs de Test.

# Toutes les prédictions sont égales à 0 (Setosa).
```

```
dummy_acc = accuracy_score(y_test, y_pred_dummy)

# Calcule l'accuracy de DummyClassifier sur Test.

# Résultat attendu : 0.3333.

print(f"DummyClassifier Accuracy : {dummy_acc:.2%}")

# Affiche : DummyClassifier Accuracy : 33.33%.

print("Same accuracy :", np.isclose(numpy_acc, dummy_acc))

# np.isclose() : vérifie si les deux accuracies sont

# numériquement égales, avec une petite tolérance.

# Résultat attendu : True.


print("Same predictions :", np.array_equal(

    y_pred_numpy, y_pred_dummy

))

# np.array_equal() : compare les deux tableaux de prédictions.

# Vérifie que les 45 prédictions sont identiques.

# Résultat attendu : True.
```

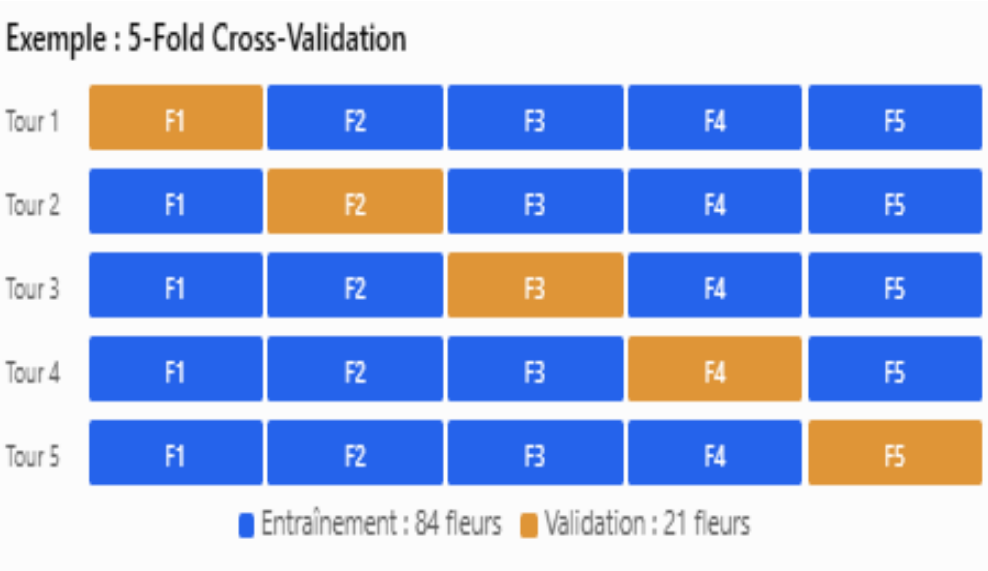
```
Classes : [0 1 2]
Counts : [35 35 35]
Majority class : 0
Predictions : [0 0 0 
```

Nous allons comparer 20 modèles kNN différents, chacun utilisant un nombre de voisins compris entre 1 et 20.

Comment fonctionne la validation croisée ?

Nous avons 105 fleurs dans Train.

La validation croisée à 5 folds les divise en cinq groupes de 21 fleurs.




```
# C2 — CHOOSING K WITH CROSS-VALIDATION

ETAPE 1. PREPARE CROSS-VALIDATION

from sklearn.model_selection import cross_val_score, StratifiedKFold

X_iris = df[features].to_numpy()

# Récupère les 4 caractéristiques des 150 fleurs Iris.

y_iris = df["target"].to_numpy()

# Récupère les classes réelles des fleurs Iris.

X_train_iris, X_test_iris, y_train_iris, y_test_iris = train_test_split(

    X_iris, y_iris,

    test_size=0.3,

    random_state=SEED,

    stratify=y_iris

)

# Sépare Iris en 70 % Train et 30 % Test.

# Train : 105 fleurs.

# Test : 45 fleurs.

# Les noms spécifiques évitent la confusion avec Wine.


cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)

# Crée une validation croisée stratifiée à 5 folds.

# shuffle=True : mélange les observations.

# random_state=SEED : garantit la reproductibilité.

k_values = range(1, 21)
```

```
# Génère les valeurs de k de 1 à 20 inclus.

print("X_train Iris :", X_train_iris.shape)

# Résultat attendu : (105, 4).

print("Nombre de valeurs de k :", len(k_values))

print("Nombre de folds :", cv.n_splits)
```

⇨ Question : Comment choisir le meilleur `k` pour kNN ?

Réponse : tester `k` de 1 à 20 avec une validation croisée à 5 folds sur Train, sélectionner le meilleur score moyen et évaluer ensuite sur Test. La moyenne des cinq accuracies est enregistrée dans `mean_scores`.

ÉTAPE 2: TEST K VALUES FROM 1 TO 20

```
mean_scores = []

# Crée une liste vide pour stocker les scores.

# Évite les doublons en cas de réexécution.

for k in k_values:

    # Parcourt les valeurs de k de 1 à 20.

    # La boucle s'exécute 20 fois.

    model = KNeighborsClassifier(n_neighbors=k)

    # Crée un modèle kNN utilisant k voisins.

    scores = cross_val_score(

        model, X_train_iris, y_train_iris,

        cv=cv, scoring="accuracy"

    )

    # Effectue une validation croisée à 5 folds.

    # Utilise uniquement les 105 fleurs de Train Iris.

    # Calcule une accuracy pour chaque fold.

    # Les données Test ne sont pas utilisées.

    mean_scores.append(scores.mean())

    # Calcule la moyenne des cinq accuracies.

    # Enregistre cette moyenne dans mean_scores.

    print(f"k={k:2d} | Mean CV accuracy={scores.mean():.2%}")

    # Affiche la valeur de k et son accuracy moyenne.

    # :2d : réserve deux caractères pour l'entier.

    # :.2% : affiche le résultat en pourcentage.

print("Nombre de scores :", len(mean_scores))

# Résultat attendu : 20.
```

```
k= 1 | Mean CV accuracy=96.19%
k= 2 | Mean CV accuracy=96.19%
k= 3 | Mean CV accuracy=96.19%
k= 4 | Mean CV accuracy=94.29%
k= 5 | Mean CV accuracy=96.19%
k= 6 | Mean CV accuracy=96.19%
k= 7 | Mean CV accuracy=97.14%
k= 8 | Mean CV accuracy=97.14%
k= 9 | Mean CV accuracy=97.14%
k=10 | Mean CV accuracy=95.24%
k=11 | Mean CV accuracy=97.14%
k=12 | Mean CV accuracy=95.24%
k=13 | Mean CV accuracy=97.14%
k=14 | Mean CV accuracy=94.29%
k=15 | Mean CV accuracy=94.29%
k=16 | Mean CV accuracy=93.33%
k=17 | Mean CV accuracy=93.33%
k=18 | Mean CV accuracy=94.29%
k=19 | Mean CV accuracy=95.24%
k=20 | Mean CV accuracy=96.19%
Nombre de scores : 20
```

Étape 3 — SELECT THE BEST K

Objectif : identifier la valeur de `k` qui possède la meilleure accuracy moyenne.

C2 — ÉTAPE 3 : SELECT THE BEST K

```
best_index = np.argmax(mean_scores)

# np.argmax() : retourne l'indice du score maximal.

# Si plusieurs scores sont égaux, retourne le premier indice.

# Ici, le premier maximum correspond à k=7.

best_k = list(k_values)[best_index]

# Récupère la valeur de k correspondant au meilleur score.

# Résultat attendu : 7.

best_cv_accuracy = mean_scores[best_index]

# Récupère la meilleure accuracy moyenne.

# Résultat attendu : environ 0.9714.

print("Best k :", best_k)

# Affiche : Best k : 7.

print(f"Best CV Accuracy : {best_cv_accuracy:.2%}")
```

```
Best k : 11
Best CV Accuracy : 97.14%
```

À retenir : le meilleur `k` est choisi à partir de l'accuracy moyenne de validation croisée sur Train, sans utiliser l'ensemble Test.

Étape 4 : Tracer le graphique :

ÉTAPE 4 : PLOT CROSS-VALIDATION RESULTS

```
plt.figure(figsize=(10, 5))

# Crée une figure de largeur 10 et hauteur 5 pouces.
```

```
plt.plot(k_values, mean_scores, marker="o")

# Trace l'accuracy moyenne pour chaque valeur de k.

# marker="o" : représente chaque résultat par un point.

plt.axvline(best_k, linestyle="--", label=f"Best k = {best_k}")

# Trace une ligne verticale au meilleur k.

# Ici, best_k = 11.

plt.xlabel("Number of neighbors (k)")

# Définit le nom de l'axe horizontal.

plt.ylabel("Mean Cross-Validation Accuracy")

# Définit le nom de l'axe vertical.

plt.title("Iris — 5-Fold Cross-Validation")

# Ajoute le titre du graphique.

plt.xticks(list(k_values))

# Affiche les valeurs entières de 1 à 20.

plt.grid(True)

# Affiche une grille facilitant la lecture.

plt.legend()

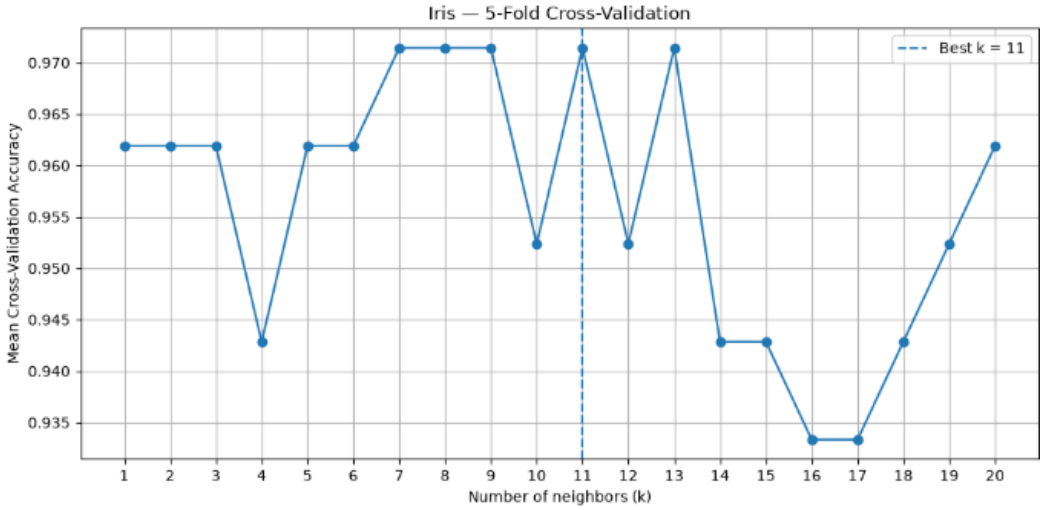
# Affiche la légende.

plt.tight_layout()

# Ajuste les espacements du graphique.

plt.show()

# Affiche le graphique.
```



Analyse des résultats

Pour k = 1 à 3 : l'accuracy moyenne est d'environ 96,19 %. Le modèle obtient déjà de bonnes performances.

Pour k = 7 à 9 : l'accuracy atteint environ 97,14 %, ce qui correspond au meilleur niveau observé.

Pour k = 11 et k = 13 : l'accuracy atteint également environ 97,14 %.

Pour k = 16 et k = 17 : l'accuracy diminue à environ 93,33 %. Ces valeurs sont moins performantes sur les folds de validation.

Même si plusieurs scores s'affichent à 97,14 %, leurs valeurs réelles peuvent différer très légèrement à cause de la précision des nombres flottants.

Le programme a donc sélectionné `k=11`.

Cela ne signifie pas que `k=7`, `k=8` ou `k=9` sont nécessairement moins bons en pratique : leurs performances observées sont quasiment identiques

À retenir

La Cross-Validation permet de choisir le meilleur `k` à partir des données d'entraînement, sans utiliser Test.

Étape 5 : Final Evaluation on Test

Maintenant, nous allons entraîner le modèle final sur les 105 fleurs de Train, puis l'évaluer une seule fois sur les 45 fleurs de Test.

Le code :

ÉTAPE 5 : FINAL EVALUATION ON TEST

```
best_model = KNeighborsClassifier(n_neighbors=best_k)

# Crée le modèle kNN final.

# n_neighbors=best_k : utilise la meilleure valeur de k

# sélectionnée par Cross-Validation.

# Dans notre cas, best_k = 11.

best_model.fit(X_train_iris, y_train_iris)

# Entraîne le modèle final sur les 105 fleurs de Train.

# X_train_iris : caractéristiques des fleurs.

# y_train_iris : classes réelles des fleurs.

# Toutes les données Train sont utilisées pour l'entraînement.

final_test_accuracy = best_model.score(X_test_iris, y_test_iris)

# Évalue le modèle final sur les 45 fleurs de Test.

# X_test_iris : caractéristiques des fleurs non vues.

# y_test_iris : classes réelles de ces fleurs.

# score() : calcule l'accuracy sur Test.

# Test n'a pas été utilisé pour choisir k.

print("Best k :", best_k)

# Affiche le meilleur k : 11.

print(f"Mean CV Accuracy : {best_cv_accuracy:.2%}")

# Affiche l'accuracy moyenne obtenue en validation croisée.

# Résultat attendu : 97.14%.

print(f"Final Test Accuracy : {final_test_accuracy:.2%}")

# Affiche l'accuracy finale du modèle sur Test.
```

```
Best k : 11
Mean CV Accuracy : 97.14%
Final Test Accuracy : 93.33%
```

Cette étape est-elle importante car elle permet de vérifier si le modèle sélectionné grâce à la Cross-Validation fonctionne également sur des données qui n'ont pas été utilisées pour choisir son hyperparamètre `k`.

C3 — StandardScaler on Wine

1. Question du TP (EN)

Put a StandardScaler before kNN on Wine with make_pipeline(...). What happens to the accuracy? Does it support your Q6 hypothesis?

Traduction (FR)

Appliquer StandardScaler avant kNN sur le dataset Wine à l'aide de make_pipeline(). Comment évolue l'accuracy ? Les résultats confirment-ils l'hypothèse formulée dans Q6 ?

2. Objectif du Challenge C3

L'objectif est d'étudier expérimentalement l'influence de la standardisation des caractéristiques sur les performances de l'algorithme kNN.

Nous allons comparer deux modèles :

- **kNN sans standardisation. On a déjà fait dans TASK6**
- **kNN avec StandardScaler.**

Les deux modèles utiliseront k=3 et seront évalués sur le même ensemble Test.

3. **Question : Pourquoi standardiser Wine avant d'utiliser kNN ?**

Réponse : pour rendre les échelles des caractéristiques comparables et éviter que des variables comme `proline` dominent les distances euclidiennes. On vérifie ensuite si l'accuracy s'améliore,car les caractéristiques du dataset Wine possèdent des échelles numériques très différentes.

Caractéristique	Valeurs approximatives
hue	0,48 à 1,71
magnesium	70 à 162
proline	278 à 1680

Sans standardisation, les caractéristiques ayant de grandes valeurs, alors StandardScaler permet de rendre les échelles comparables en standardisant chaque caractéristique à partir de la moyenne et de l'écart-type calculés sur Train.

C3 — Étape 1 : Préparer les données Wine :

1. Objectif de cette étape

Charger le dataset Wine et séparer les observations en deux ensembles :

- Train (70 %) : données utilisées pour entraîner le modèle.
- Test (30 %) : données réservées pour évaluer le modèle.

Nous conservons SEED=42 et la stratification utilisés dans la Task 6, afin de comparer les modèles sur le même découpage.

2. Code Python — Prepare Wine Data

Ajoute une nouvelle cellule Python dans ton notebook Jupyter.

Le code :

STEP 1 : PREPARE WINE DATA

```
import numpy as np
# Importe NumPy pour manipuler les tableaux.
from sklearn.datasets import load_wine
# Importe la fonction permettant de charger Wine.
from sklearn.model_selection import train_test_split
# Importe la fonction de séparation Train/Test.
SEED = 42
# Fixe la graine aléatoire pour assurer la reproductibilité.
wine = load_wine(as_frame=True)
# Charge le dataset Wine contenant 178 observations.
X_wine = wine.data.to_numpy()
# Récupère les 13 caractéristiques chimiques des vins.
y_wine = wine.target.to_numpy()
# Récupère les classes réelles des vins (0, 1 ou 2).
X_train_wine, X_test_wine, y_train_wine, y_test_wine = train_test_split(
    X_wine, y_wine,
    test_size=0.3,
    random_state=SEED,
    stratify=y_wine
)
# Divise les données en 70 % Train et 30 % Test.
# random_state=SEED : rend le découpage reproductible.
# stratify=y_wine : conserve les proportions des classes.
print("Train :", X_train_wine.shape)
# Affiche les dimensions de l'ensemble Train.
print("Test :", X_test_wine.shape)
# Affiche les dimensions de l'ensemble Test.
print("Train class counts :", np.bincount(y_train_wine))
# Affiche le nombre de vins de chaque classe dans Train.
print("Test class counts :", np.bincount(y_test_wine))
# Affiche le nombre de vins de chaque classe dans Test.
```

```
Train : (124, 13)
Test : (54, 13)
Train class counts : [41 50 33]
Test class counts : [18 21 15]
```

C3 — Étape 2 : kNN WITHOUT STANDARDIZATION

1. Objectif de cette étape

Nous allons entraîner un modèle kNN avec k=3, sans appliquer StandardScaler.

L'objectif est de retrouver l'accuracy de 68,52 %, déjà obtenue dans la Task 6 (Q6). Cette valeur servira de référence pour mesurer ensuite l'effet de la standardisation.

Le code :

```
 STEP 2 : KNN WITHOUT STANDARDIZATION

from sklearn.neighbors import KNeighborsClassifier

# Importe l'algorithme de classification kNN.

knn_without_scaling = KNeighborsClassifier(n_neighbors=3)

# Crée un classificateur kNN avec 3 voisins.

# Les caractéristiques ne sont pas standardisées.

knn_without_scaling.fit(X_train_wine, y_train_wine)

# Entraîne le modèle sur les 124 vins de Train.

# X_train_wine : les 13 caractéristiques des vins.

# y_train_wine : les classes réelles des vins.

accuracy_without_scaling = knn_without_scaling.score(

    X_test_wine, y_test_wine

)

# Évalue le modèle sur les 54 vins de Test.

# score() calcule la proportion de prédictions correctes.

print(f"Accuracy without scaling : {accuracy_without_scaling:.2%}")

# Affiche l'accuracy du modèle sans standardisation.
```

```
Accuracy without scaling : 68.52%
```

Comment interpréter ce résultat ?

Une accuracy de 68,52 %(exactement comme dans la Task 6 (Q6)) signifie que le modèle classe correctement environ 68,52 % des 54 vins de Test, en utilisant directement les caractéristiques dans leurs échelles d'origine.

Nous ne pouvons pas encore conclure que la standardisation améliorera les performances. Pour le vérifier, il faudra comparer ce résultat avec celui d'un deuxième modèle, entraîné sur les mêmes données mais **avec StandardScaler.**

Étape 3 : kNN WITH STANDARDIZATION

1. Objectif de cette étape

Appliquer StandardScaler avant kNN à l'aide de make_pipeline() afin de rendre les échelles des 13 caractéristiques comparables.

Nous conservons :

- Le même dataset Wine.
- Les mêmes ensembles Train et Test.
- Le même nombre de voisins : k=3.

Seule la standardisation change.

2. Pourquoi utiliser `make_pipeline()` ?

Un pipeline exécute automatiquement deux opérations successives :

Wine Train → StandardScaler → kNN (k=3)

La standardisation est apprise uniquement sur Train, puis appliquée à Test avec les mêmes paramètres.

Cela permet d'éviter le data leakage (fuite d'informations).

STEP 3 : KNN WITH STANDARDIZATION

```
from sklearn.preprocessing import StandardScaler

# Importe la classe permettant de standardiser les données.

from sklearn.pipeline import make_pipeline

# Importe la fonction permettant de créer un pipeline.

knn_with_scaling = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=3)
)

# Crée un pipeline en deux étapes :

# 1. StandardScaler() : standardise les caractéristiques.

# 2. KNeighborsClassifier() : applique kNN avec k=3.

knn_with_scaling.fit(X_train_wine, y_train_wine)

# Calcule les paramètres de standardisation sur Train.

# Transforme Train puis entraîne le modèle kNN.

accuracy_with_scaling = knn_with_scaling.score(
    X_test_wine, y_test_wine
)

# Standardise Test à partir des paramètres appris sur Train.

# Calcule l'accuracy du modèle sur les 54 vins de Test.

print(f"Accuracy with scaling : {accuracy_with_scaling:.2%}")

# Affiche l'accuracy obtenue après standardisation.
```

```
Accuracy with scaling : 94.44%
```

Étape 4 : Compare Results

1. Objectif

Calculer l'amélioration de l'accuracy après standardisation.

STEP 4 : COMPARE RESULTS

```
improvement = accuracy_with_scaling - accuracy_without_scaling

# Calcule la différence entre les deux accuracies.

print(f"Without scaling : {accuracy_without_scaling:.2%}")

# Affiche l'accuracy sans standardisation.

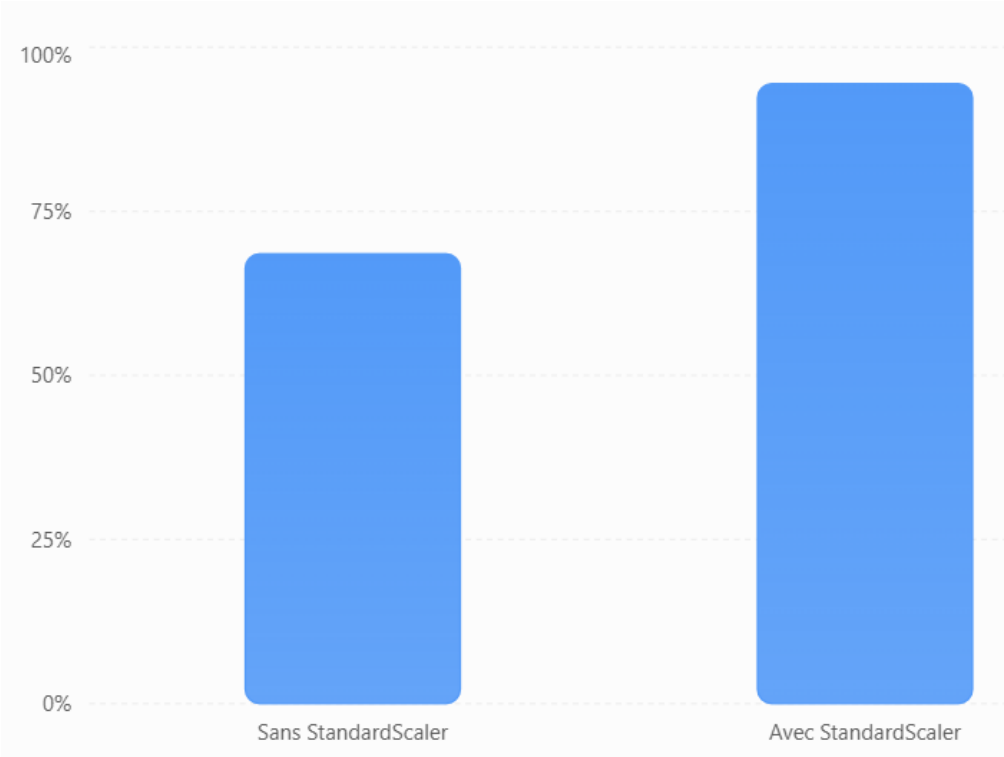
print(f"With scaling    : {accuracy_with_scaling:.2%}")

# Affiche l'accuracy avec standardisation.

print(f"Improvement     : {improvement * 100:.2f} percentage points")

# Convertit la différence en points de pourcentage.
```

```
Without scaling : 68.52%
With scaling    : 94.44%
Improvement     : 25.93 percentage points
```



⇨ La standardisation améliore l'accuracy de kNN sur Wine de 68,52 % à 94,44 %, soit une augmentation d'environ 25,93 points de pourcentage. Ce résultat confirme notre hypothèse de Q6 selon laquelle les différences d'échelle influencent les algorithmes basés sur les distances. StandardScaler évite que les caractéristiques ayant de grandes amplitudes numériques, comme proline, dominent les distances euclidiennes.

C4 — Étape 1 : Comprendre l'objectif

1. Question officielle du TP (EN)

Time sum(v * v for v in data) against np.dot(data, data) on one million random numbers using time.perf_counter().

Traduction (FR)

Comparer le temps d'exécution d'une boucle Python sum(v * v for v in data) avec l'opération vectorisée NumPy np.dot(data, data), sur un million de nombres aléatoires, en utilisant time.perf_counter().

2. Objectif du Challenge C4

L'objectif est de **comparer les performances** d'une boucle Python et d'une opération vectorisée NumPy.

Les deux méthodes calculent la même opération : la somme des carrés de tous les éléments d'un tableau.

$$S = x_1^2 + x_2^2 + \cdots + x_{1\,000\,000}^2$$

STEP 1 : CREATE DATA

```
import numpy as np

# Importe NumPy pour manipuler les tableaux numériques.

import time

# Importe le module permettant de mesurer le temps.

rng = np.random.default_rng(42)

# Crée un générateur de nombres aléatoires reproductible.

data = rng.random(1_000_000)

# Génère un million de nombres aléatoires entre 0 et 1.

# Le caractère _ facilite la lecture du nombre 1000000.

print("Data shape :", data.shape)

# Affiche les dimensions du tableau.

print("Data type :", data.dtype)

# Affiche le type de données des éléments.
```

```
Data shape : (1000000,)
Data type : float64
```

Cela signifie que ton tableau data contient 1 000 000 de valeurs disposées sur une seule dimension.

- 1000000 : nombre d'éléments du tableau.
- La virgule , : indique que le tableau possède une seule dimension.
- shape : indique les dimensions du tableau NumPy.

⇨ Le tableau contient un million de nombres réels stockés dans un tableau NumPy à une dimension. Chaque élément est représenté avec le type float64, utilisant 64 bits.

C4 — Étape 2 : Python Loop

1. Objectif

Calculer la somme des carrés des 1 million de nombres avec Python, puis mesurer le temps d'exécution.

Nous utiliserons deux fonctions :

- sum() : additionne les valeurs.
- time.perf_counter() : mesure précisément le temps écoulé.

Le code :

```
STEP 2 : PYTHON LOOP

start_python = time.perf_counter()

# Enregistre l'instant où commence le calcul.

result_python = sum(v * v for v in data)

# Parcourt les 1 million de valeurs de data.

# v représente successivement chaque élément.

# v * v calcule le carré de chaque élément.

# sum() additionne tous les carrés.

end_python = time.perf_counter()

# Enregistre l'instant où se termine le calcul.

time_python = end_python - start_python

# Calcule le temps écoulé en secondes.

print(f"Python result : {result_python:.6f}")

# Affiche la somme des carrés avec 6 décimales.

print(f"Python time   : {time_python:.6f} seconds")

# Affiche le temps nécessaire au calcul.
```

```
Python result : 333336.884885
Python time   : 0.115131 seconds
```

⇨ **Interprétation :**

333336.884885 : somme des carrés des 1 000 000 de nombres aléatoires.

0.115131 seconde : temps nécessaire à Python pour effectuer ce calcul, soit environ 115 millisecondes.

Nous allons maintenant effectuer exactement le même calcul avec NumPy afin de comparer les performances.

C4 — Étape 3 : NumPy Vectorization

Objectif

Calculer la somme des carrés avec np.dot() et mesurer le temps d'exécution.

```
STEP 3 : NUMPY VECTORIZATION

start_numpy = time.perf_counter()

# Enregistre l'instant de départ du calcul.
```

```
result_numpy = np.dot(data, data)

# Calcule le produit scalaire du tableau avec lui-même.

# Cela équivaut à la somme des carrés de ses éléments.

# NumPy utilise des routines numériques optimisées.

end_numpy = time.perf_counter()

# Enregistre l'instant de fin du calcul.

time_numpy = end_numpy - start_numpy

# Calcule le temps d'exécution en secondes.

print(f"NumPy result : {result_numpy:.6f}")

# Affiche la somme des carrés avec 6 décimales.

print(f"NumPy time   : {time_numpy:.6f} seconds")

# Affiche le temps nécessaire au calcul NumPy.
```

⇨ Le résultat numérique est identique à celui de la boucle Python à six décimales, mais NumPy a effectué le calcul beaucoup plus rapidement.

C4 — Étape 4 : Compare Performance

1. Question (EN)

How can we compare the execution times of a Python loop and NumPy vectorization?

Traduction (FR)

Comment comparer les temps d'exécution d'une boucle Python et de la vectorisation NumPy ?

Question pédagogique dérivée de la consigne officielle C4.

2. Objectif

Comparer les résultats numériques, les temps d'exécution et calculer le Speedup (facteur d'accélération).

```
 STEP 4 : COMPARE PERFORMANCE

same_result = np.isclose(result_python, result_numpy)

# Vérifie que les deux résultats numériques sont proches.

# Accepte les petites différences d'arrondi.

speedup = time_python / time_numpy

# Calcule le rapport des temps d'exécution.

# Si speedup > 1, NumPy est plus rapide.

print("Same numerical result :", same_result)

# Affiche True si les résultats sont équivalents.

print(f"Python loop time : {time_python:.6f} s")

# Affiche le temps de la boucle Python.
```

```
print(f"NumPy time       : {time_numpy:.6f} s")

# Affiche le temps de NumPy.

print(f"Speedup          : {speedup:.2f}x")

# Affiche le facteur d'accélération.
```

```
Same numerical result : True
Python loop time : 0.115131 s
NumPy time       : 0.001861 s
Speedup          : 61.87x
```

$$\text{Speedup} = \frac{0.115131}{0.001861} \approx 61.86$$

⇨ NumPy est donc environ 61,9 fois plus rapide que la boucle Python dans cette mesure.

Attention : les temps peuvent varier d'une exécution à l'autre. Une comparaison plus fiable nécessite plusieurs répétitions.

⇨ Que signifie `Same numerical result : True` ?

Cela signifie que la boucle Python et `np.dot()` donnent des résultats numériquement équivalents.

Les deux méthodes calculent la même somme des carrés :

$$S = \sum_{i=1}^{1\,000\,000} x_i^2$$

⇨ Que signifie Speedup : 61.87x ?

Cela signifie que, pendant cette mesure, NumPy a été 61,87 fois plus rapide que la boucle Python.

La raison principale est que NumPy exécute des opérations numériques avec des routines compilées optimisées, alors qu'une boucle Python traite successivement les éléments au niveau de l'interpréteur.

On conclue par ; Les deux méthodes produisent des résultats numériquement équivalents. La boucle Python prend 0,115131 seconde, tandis que NumPy ne prend que 0,001861 seconde.

Dans cette expérience, NumPy est environ 61,87 fois plus rapide, car il utilise des routines numériques compilées et optimisées au lieu d'exécuter individuellement chaque opération dans l'interpréteur Python.

à retenir

Data Leakage — Fuite de données en Machine Learning

Le Data Leakage (fuite de données) se produit lorsque des informations qui ne devraient pas être accessibles pendant l'entraînement ou la sélection du modèle sont utilisées, directement ou indirectement, ce qui peut produire des performances artificiellement élevées.